PurePy

Language Specification, v0.15.1

Copyright © 2026 PurePy contributors. Distributed under the MIT License.

Contents

Summary **3**
Judgement forms . . . 3
Auxiliary functions . . . 3

1 Conformance **5**
1.1 Conformance suite . . . 5

2 Syntax **6**
2.1 Lexical syntax . . . 6
2.2 Contexts . . . 6
2.3 Modules and programs . . . 8

3 Well-formedness **13**
3.1 Auxiliary definitions . . . 13
3.2 Statements, expressions and patterns . . . 13
3.3 Modules and programs . . . 14

4 Operational semantics **20**
4.1 Values and environments . . . 20
4.2 Auxiliary definitions . . . 20
4.3 Pattern matching . . . 20
4.4 Statements and expressions . . . 21
4.5 Modules and programs . . . 24

A Auxiliary functions (normative) **31**
A.1 Syntactic analyses . . . 31
A.2 Classes and modules . . . 32
A.3 Values . . . 34

B Excluded Python behaviours (informative) **38**
B.1 Mutable state . . . 38
B.2 Implicit coercions . . . 38
B.3 Runtime errors . . . 39

Summary

Judgement forms

The judgement forms of the well-formedness rules and the operational semantics, with the figure defining each.

Judgement	Reading	Defined in
Well-formedness		
$\Gamma \vdash p_1 \cdot \ldots \cdot p_{n+1}$	patterns of a `match` are linear and each reachable	Figure 3.4
$\Gamma \vdash p \sqsubseteq p'$	every value matched by p is matched by p'	Figure 3.3
$\Gamma \vdash q : \theta$	qualified name q denotes entry θ	Figure 3.2
$\Gamma \vdash e$	expression e is well-formed	Figure 3.2
$\Gamma \vdash \vec{g} : \Delta$	comprehension qualifiers $\vec{g}$ are well-formed and bind Δ	Figure 3.2
$\Gamma \vdash t : R$	statement t is well-formed with result type R	Figure 3.1
Γ, x imports θ	name x imported from module signature Γ has entry θ	Figure 3.5
$\vdash \vec{\imath} : \Gamma$	imports $\vec{\imath}$ are well-formed and yield context Γ	Figure 3.6
q, θ loads-as θ'	importing module q into entry θ gives entry θ'	Figure 3.6
$\vdash_{\mathsf{load}} q : \Gamma$	module q statically loads with signature Γ	Figure 3.7
$\vdash \mathcal{M}$	program $\mathcal{M}$ is well-formed	Figure 3.7
Evaluation		
$\rho, p, v \rightsquigarrow m$	pattern p matches value v with result m	Figure 4.2
$\rho, t \Rightarrow r$	statement t evaluates to result r	Figure 4.3
$\rho, e \Rightarrow o$	expression e evaluates to outcome o	Figure 4.5
$\rho, \vec{e} \Rightarrow o$	expressions $\vec{e}$ evaluate, left to right, to outcome o	Figure 4.7
$\rho, \vec{g} \Rightarrow o$	comprehension qualifiers $\vec{g}$ evaluate to outcome o	Figure 4.10
ρ, x imports o	name x imported from module environment ρ has outcome o	Figure 4.11
$\vec{\imath} \Rightarrow o$	imports $\vec{\imath}$ evaluate to outcome o	Figure 4.12
q, v loads-as v'	importing module q into value v gives value v'	Figure 4.12
$q \Rightarrow_{\mathsf{load}} o$	module q loads to outcome o	Figure 4.13
$\mathcal{M} \Rightarrow \kappa$	program $\mathcal{M}$ evaluates to termination kind κ	Figure 4.13

Auxiliary functions

The auxiliary functions used by the rules, with the section defining each.

Function	Result	Meaning	Section
Syntactic analyses			
assigns(t)	set of variables	variables assigned in t, not descending into nested functions	§A.1
binds(p), binds($\vec{g}$)	set of variables	variables bound by pattern p, or by the generators of qualifiers $\vec{g}$	§A.1
captures(t), captures($\vec{g}$)	set of variables	variables of enclosing scope captured by lambdas and nested functions in t, or by lambdas in qualifiers $\vec{g}$	§A.1
captures$_e$(e)	set of variables	variables of enclosing scope captured by lambdas in e	§A.1
fv(s)	set of variables	free variables of statement s	§A.1
fv$_e$(e)	set of variables	free variables of expression e	§A.1
Classes and modules			
submods(q)	context	stub for each submodule of q	§A.2
own-fields(ϕ)	sequence of variables	field names declared by field list ϕ	§A.2
fields(C)	sequence of variables	fields of class C, inherited first	§A.2
ancestors(C)	set of class entries	C and its base classes	§A.2
field-map(C, $\vec{u}$)	map from variables	argument supplied for each field of C; undefined unless arguments cover fields exactly	§A.2
resolve-class(ρ, q)	class entry	class denoted by qualified name q in ρ	§A.2
env(ρ, q)	environment	environment of module q as loaded in ρ	§A.2
name(q)	string	string literal naming module q	§A.2
Values			
eq(v, v')	Boolean	structural equality; undefined between unrelated values	§A.3
eq-elems($\vec{v}$, $\vec{v}'$)	Boolean	elementwise equality of value sequences	§A.3
contains(v, v')	Boolean	whether v' is an element of v	§A.3
contains-elems($\vec{v}$, v)	Boolean	whether $\vec{v}$ has an element equal to v	§A.3
elems(v)	sequence of values	elements of list, tuple or string	§A.3
iter(v)	sequence of values	sequence a generator draws from v; keys if v is a dictionary	§A.3
getitem(v, v')	value or abort	subscripting; aborts with IndexError for an index out of range, KeyError for a missing key and TypeError otherwise	§A.3
outcome(r)	value or abort	statement result read as an outcome: the returned value, `None` if the block falls off the end, an abort unchanged	§A.3
dispatch(ρ, v, $\vec{p}$, $\vec{s}$)	bindings and statement	case taken by a `match`	§A.3
entries(δ, δ')	dictionary entries	δ extended by entries of δ' in order, later overriding earlier	§A.3
update(δ, w, v)	dictionary entries	δ with w bound to v, replacing any earlier binding	§A.3

1 Conformance

This document defines PurePy version 0.15.1. PurePy is a subset of Python: every well-formed program is a syntactically valid Python program, and if PurePy gives a run a result, that run produces the same result under Python. Python means Python 3.12 throughout. Chapter 2 gives the abstract syntax; the lexical syntax is that of Python (§2.1). Chapter 3 defines well-formedness, the static part of the definition, and Chapter 4 the operational semantics, which is defined for well-formed programs. A run is *valid* when the semantics assigns it a result; any other run reaches an operation the semantics leaves undefined.

Well-formedness is a property of the program, decided by PurePy's static checks. Semantic validity is a property of a run rather than a program, because some of the Python behaviours PurePy excludes cannot be rejected statically (summarised in the informative Annex B). An implementation conforms if it accepts exactly the well-formed programs and, on every valid run, produces the Python result; on any other run it may abort at the dynamically excluded operation, or simply produce the Python result as a stock interpreter does. The conformance suite (§1.1) organises its tests by verdict: semantically valid PurePy, excluded by PurePy (syntactically, statically or dynamically), or failing under Python.

1.1 Conformance suite

The conformance suite pairs each test with the verdict Python itself gives, and runs on CPython, GraalPy, and Pyodide. [**TODO:** Diagram showing the various kinds of test.] [**TODO:** Reference interpreter, needed to detect dynamically excluded operations (#64).]

2 Syntax

The syntax of PurePy is given in Figures 2.3, 2.4 and 2.5. Statements are sequenced by juxtaposition, $s\ s'$; a function body, a branch of a conditional and the body of a case are statements. A *top-level statement* t is a statement or a class declaration, sequenced likewise; the body of a module after its imports is a top-level statement, so class declarations occur only at the top level of a module. A *mutual region* $d_1 \dots d_{n+1}$ is a maximal run of consecutive `def` statements in a sequence: any other statement separates mutual regions. Maximality is not enforced by the grammar in Figure 2.3; the parser is responsible for grouping consecutive `def`s into a single region. The well-formedness rules treat each region as one mutually-recursive binding. Figure 2.1 summarises the metavariables used throughout.

2.1 Lexical syntax

The grammar of PurePy is an abstract syntax, and its metavariables range over values rather than over source text: n is a number and w is a string, not the lexemes that denote them. How source text becomes abstract syntax follows Python, including the indentation that delimits blocks.

2.2 Contexts

Metavariables x, f, c range over *variables*, simple identifiers whatever they denote; idiomatically f is used when the variable is expected to denote a function and c when it is expected to denote a class. A *qualified name* q is a non-empty sequence of variables separated by dots (Figure 2.3); a variable is thus a qualified name with a single component.

Definition 2.1 (Context)**.** A context Γ (or Δ) is a finite map from variables to one of the following *context entries* θ:

- the variable's *definite assignment* status $a \in \mathbb{B} = \{\mathsf{tt}, \mathsf{ff}\}$;
- a *module reference*, for a fully-qualified module name q: either a *stub* $\mathsf{Mod}(q)$, or *loaded* reference $\mathsf{Mod}(q, \Gamma)$ carrying the *signature* Γ of q, the context specifying its members;
- a *class entry* C, of the form $\mathsf{Cls}_\Gamma(q.c, \vec{x}, c')$, recording the declaring context Γ, the qualified name $q.c$ of the class, sequence of distinct field names $\vec{x}$, and base class name c' (or $\bot$ for no base).

Module references arise from import statements. A module declares each class name at most once, so the qualified name of a class entry is the identity of the class. The definite-assignment terminology is borrowed from Java [1, §16]; tt indicates *definitely assigned* and ff *not definitely assigned* (or, equivalently, degenerate types "any" and "none"). It is useful to represent contexts as total functions on the set of all variables by extending the codomain with $\bot$ to indicate *undefined*. We write $\Gamma, x : \theta$ for the context that sends x to θ and otherwise agrees with Γ, define $\mathrm{dom}(\Gamma) = \{x \mid \Gamma(x) \neq \bot\}$, and write $\emptyset$ for the context where all variables are undefined. For a set of variables X, write $\mathsf{tt}(X)$ for the context $\{x : \mathsf{tt} \mid x \in X\}$; similarly $\mathsf{ff}(X)$.

Names and literals		
x, f, c	variable; idiomatically f a function and c a class	§2.2
q	qualified name of a module or class	Figure 2.3
n	number; also a sequence length and an exit status	Figure 2.4
w	string	Figure 2.4
ℓ	literal	Figure 2.4
$\odot$, $\ominus$	binary and unary operator	Figure 2.6
Syntax		
e	expression	Figure 2.4
p	pattern	Figure 2.4
g	comprehension qualifier	Figure 2.4
s	statement	Figure 2.3
d	function definition	Figure 2.3
τ, σ	type	Figure 2.5
π	optional annotation	Figure 2.5
ω	optional return annotation	Figure 2.5
ι	import	Figure 2.3
ϕ	field list of a dataclass	Figure 2.3
t	top-level statement	Figure 2.3
μ	module body	Figure 2.3
u	argument: an expression, a pattern or a value	Chapter 2
Programs		
$\mathcal{M}$	program, a finite map from module names to bodies	§2.3
$\mathcal{P}$	predefined modules, each with its environment and context	§2.3
Contexts		
Γ, Δ	context, a finite map from variables to entries	Definition 2.1
θ	context entry	Definition 2.1
a	definite-assignment status, tt or ff	Definition 2.1
C	class entry	Definition 2.1
R	result type	§3.2
Values and results		
v	value	Figure 4.1
ρ	environment, a finite map from variables to values	Figure 4.1
δ	dictionary entries	Figure 4.1
r	result of statement evaluation	Figure 4.1
o	outcome of expression evaluation	Figure 4.1
m	match result	Figure 4.1
κ	termination kind	Figure 4.1
f	meaning of a primitive, a partial map from a sequence of values to an outcome	Figure 4.1

Figure 2.1: Metavariables used throughout the specification

Context operations. Contexts admit a non-commutative monoid of sequential composition $\Gamma \otimes \Delta$ ("override") and an idempotent commutative semigroup of parallel composition $\oplus$ ("merge"). Together these form a *right near-semiring* [2]: $\otimes$ right-distributes over $\oplus$, but left distributivity does not hold. Both are defined pointwise (Figure 2.2): override is right-biased and has $\bot$ as both left and right unit; merge is defined only on definite-assignment statuses, returning ff when exactly one argument is defined and the conjunction otherwise, so $\bot$ is not a unit.

$\otimes$ (override)

$$\bot \otimes \theta = \theta \otimes \bot = \theta$$
$$\theta \otimes \theta' = \theta' \quad \text{(otherwise)}$$

$\oplus$ (merge)

$$\bot \oplus \bot = \bot$$
$$a \oplus \bot = \bot \oplus a = \mathsf{ff}$$
$$a \oplus a' = a \wedge a' \quad (a, a' \in \mathbb{B})$$

$\lhd$ (extension)

$$\mathsf{Mod}(q, \Gamma) \lhd \mathsf{Mod}(q, \Gamma') = \mathsf{Mod}(q, \Gamma \lhd \Gamma')$$
$$\mathsf{Mod}(q) \lhd \mathsf{Mod}(q, \Gamma) = \mathsf{Mod}(q, \Gamma) \lhd \mathsf{Mod}(q) = \mathsf{Mod}(q, \Gamma)$$
$$\bot \lhd \theta = \theta \lhd \bot = \theta$$
$$\theta \lhd \theta' = \theta' \quad \text{(otherwise)}$$

Figure 2.2: Context operations, defined on entries and lifted pointwise to contexts: $(\Gamma \cdot \Delta)(x) = \Gamma(x) \cdot \Delta(x)$

A third operation, *context extension* $\Gamma \lhd \Delta$, also Figure 2.2, combines module references for the same module recursively, preferring loaded references to stubs, and is otherwise right-biased like override. It is used when processing import prefixes, where two imports rooted at the same module must combine rather than shadow.

Operators. The metavariable $\odot$ ranges over the supported binary operators other than `and` and `or`, and $\ominus$ over the supported unary operators, both enumerated in Figure 2.6. The operators `and` and `or` have their own expression forms; they do not evaluate both operands. The grammar of Figure 2.4 is intentionally ambiguous on operator forms; disambiguation is by the precedence and associativity given in Figure 2.6, which follow Python.

Type annotations. Parameters and assignments carry optional type annotations π and function definitions optional return type annotations ω, while dataclass fields carry types τ; types and annotations are given in Figure 2.5. We write $\vec{x}\,\vec{\pi}$ for the parameter list $x_1\pi_1, \ldots, x_n\pi_n$. The well-formedness rules and operational semantics do not consult annotations, which are given meaning by the type system.

2.3 Modules and programs

A PurePy *program* is a finite map $\mathcal{M}$ from *fully-qualified module names* (Figure 2.3) to *module bodies* μ, of which one is a distinguished *main module* named `__main__`; here name(q) is the string literal naming q, so name(`__main__`) = `"__main__"`. We identify a module with its fully-qualified name, writing "module q" for the module named q. A module body is a sequence of imports $\vec{\imath}$, its *import prefix*, followed by a top-level statement t; since imports are not statements, they can appear neither within a

Statements			
s	$::=$	`pass`	(Pass)
	$\mid$	$s\ s'$	(Seq)
	$\mid$	$x\pi = e$	(Assign)
	$\mid$	e	(ExprStmt)
	$\mid$	`assert` e	(Assert)
	$\mid$	`assert` e, e'	(AssertMsg)
	$\mid$	`return`	(ReturnNone)
	$\mid$	`return` e	(Return)
	$\mid$	`if` e `:` s `elif` e_1 `:` s_1 … `elif` e_n `:` s_n	(If)
	$\mid$	`if` e `:` s `elif` e_1 `:` s_1 … `elif` e_n `:` s_n `else :` s'	(IfElse)
	$\mid$	`match` e`:` `case` p_1`:` s_1 … `case` p_{n+1}`:` s_{n+1}	(Match)
	$\mid$	d_1 … d_{n+1}	(MutualRegion)
Top-level statements			
t	$::=$	s	(Stmt)
	$\mid$	$t\ t'$	(TopSeq)
	$\mid$	`@dataclass class` c `:` ϕ	(Dataclass)
	$\mid$	`@dataclass class` $c(c')$ `:` ϕ	(DataclassExtend)
Imports			
ι	$::=$	`import` q	(Import)
	$\mid$	`from` q `import` $x_1, \ldots, x_{n+1}$	(FromImport)
Field lists			
ϕ	$::=$	x_1`:` τ_1 … x_{n+1}`:` τ_{n+1}	(FieldList)
	$\mid$	`pass`	(EmptyFieldList)
Module bodies			
μ	$::=$	ι_1 … ι_n t	(ModuleBody)
Qualified names			
q	$::=$	$x_1.x_2.\cdots.x_{n+1}$	(Name)
Function definitions			
d	$::=$	`def` $f(x_1\pi_1, \ldots, x_n\pi_n)\,\omega$`:` s	(Def)

Figure 2.3: Modules and statements

statement nor after it. The statement begins with the assignment `__name__` = name(q), which binds the module's name when it loads (§3.3).

A second map $\mathcal{P}$ gives the predefined modules of Figure 2.7, which have no body: $\mathcal{P}$ takes each such name to its environment and context (Definition 2.2). The two domains are disjoint, and their union is prefix-closed: the parent of any module in $\mathrm{dom}(\mathcal{M}) \cup \mathrm{dom}(\mathcal{P})$ is itself in $\mathrm{dom}(\mathcal{M}) \cup \mathrm{dom}(\mathcal{P})$.[1] The maps $\mathcal{M}$ and $\mathcal{P}$ are fixed and ambient in all definitions that follow, as is the name q of the enclosing module in the rules that check or evaluate a single module's body.

Definition 2.2 (Predefined contexts and environments)**.** $\mathcal{P}$ takes each module q of Figure 2.7 to a pair

[1] A source tree encodes such an $\mathcal{M}$: its modules are the `.py` files and the directories, each named by its path, so the domain is prefix-closed by construction. A module's statement is the statements of its `.py` file, or of its `__init__.py` for a directory, preceded by the `__name__` assignment; a directory without an `__init__.py` (a namespace package, PEP 420, https://peps.python.org/pep-0420/) contributes that assignment alone.

Expressions			
e	::=	x	(Var)
	|	ℓ	(Literal)
	|	$e(e_1, \ldots, e_n)$	(Call)
	|	$q(e_1, \ldots, e_n, x_{n+1}{=}e_{n+1}, \ldots, x_{n+m}{=}e_{n+m})$	(Constr)
	|	$\{e_1 : e'_1, \ldots, e_n : e'_n\}$	(Dict)
	|	$e.x$	(Attribute)
	|	$e[e']$	(Subscript)
	|	$e \odot e'$	(BinOp)
	|	$\ominus\, e$	(UnOp)
	|	e `and` e'	(And)
	|	e `or` e'	(Or)
	|	e_1 `if` e `else` e_2	(CondExpr)
	|	$[e_1, \ldots, e_n]$	(List)
	|	$(e_1, \ldots, e_n)$	(Tuple)
	|	$[\, e\ g_1\ \ldots\ g_n]$	(ListComp)
	|	$\{e : e'\ g_1\ \ldots\ g_n\}$	(DictComp)
	|	`lambda` $x_1, \ldots, x_n :\ e$	(Lambda)

Comprehension qualifiers			
g	::=	`if` e	(Guard)
	|	`for` x `in` e	(Generator)

Literals			
ℓ	::=	n	(Number)
	|	w	(String)
	|	`True`	(True)
	|	`False`	(False)
	|	`None`	(None)

Patterns			
p	::=	ℓ	(PatLiteral)
	|	$-n$	(PatNegLit)
	|	x	(PatVar)
	|	_	(PatWildcard)
	|	$[p_1, \ldots, p_n]$	(PatList)
	|	$(p_1, \ldots, p_n)$	(PatTuple)
	|	$q(p_1, \ldots, p_n, x_{n+1}{=}p_{n+1}, \ldots, x_{n+m}{=}p_{n+m})$	(PatConstr)
	|	$\{w_1 : p_1, \ldots, w_n : p_n\}$	(PatDict)
	|	p `as` x	(PatAs)

Figure 2.4: Expressions and patterns

(ρ, Γ), where ρ binds `__name__` to name(q) and each member listed for q in Figure 2.7 to its value, and Γ binds the same variables at tt.

Write $q \preceq q'$ (*prefix*) when $q' = q$ or $q' = q.q^\dagger$ for some $q^\dagger$, and $\prec$ for the strict order; the proper prefixes of a module name are its *ancestors*. The metafunctions on modules and classes are defined in §A.2.

Types			
τ	::=	`Any`	(Any)
	|	`None`	(None)
	|	`bool`	(Bool)
	|	`int`	(Int)
	|	`float`	(Float)
	|	`str`	(Str)
	|	`list`$[\tau]$	(List)
	|	`tuple`$[\tau_1, \ldots, \tau_n]$	(Tuple)
	|	`dict`$[$`str`$, \tau]$	(Dict)
	|	`Callable`$[[\tau_1, \ldots, \tau_n], \tau]$	(Callable)
	|	q	(Class)
	|	$\sigma \mid \tau$	(Union)

Annotations			
π	::=	`:` τ	(Annot)
	|	ε	(Unannot)

Return annotations			
ω	::=	`->` τ	(ReturnAnnot)
	|	ε	(ReturnUnannot)

Figure 2.5: Types and annotations

Level	Operators	Arity	Associativity
1 (lowest)	`or`	binary	left
2	`and`	binary	left
3	`not`	unary	—
4	`==` `!=` `<` `<=` `>` `>=` `in` `not in`	binary	left
5	`+` `-`	binary	left
6	`*` `/` `//` `%`	binary	left
7	`+` `-`	unary	—
8 (highest)	`**`	binary	right

Figure 2.6: Operator precedence (lowest binds loosest)

Module	Member	Value	
`builtins`	`print`		
	`len`	$\mathsf{Prim}(f)$ where	
		$f(v) = \mathsf{val}\ n$	if $\mathrm{iter}(v) = u_1 \cdot \ldots \cdot u_n$
		$f(\vec{u}) = \mathsf{aborts\ TypeError}$	(otherwise)
	`range`	$\mathsf{Prim}(f)$ where	
		$f(n) = \mathsf{val}\ [0, \ldots, n-1]$	if n whole number, $n > 0$
		$f(n) = \mathsf{val}\ []$	if n whole number, $n \leq 0$
`math`	`pi`		
	`e`		
	`sqrt`		
	`exp`		
	`log`		
	`sin`		
	`cos`		
	`tan`		
	`floor`		
	`ceil`		
`sys`	`argv`		
	`exit`	$\mathsf{Prim}(f)$ where	
		$f(\varepsilon) = \mathsf{aborts\ SystemExit}(0)$	
		$f(n) = \mathsf{aborts\ SystemExit}(n)$	
`typing`	`Any`		
	`Callable`		
`dataclasses`	`dataclass`		

Figure 2.7: Predefined modules and their members

3 Well-formedness

PurePy excludes some Python behaviours statically and others only at runtime. Well-formedness is the static part, a judgement on programs defined in this chapter; the operational semantics (Chapter 4) is defined for well-formed programs. The statement and expression rules rest on a definite-assignment analysis; the pattern rules enforce linearity and reachability; and the module rules govern imports and class declarations. The chapter follows the syntax of Chapter 2, taking statements, expressions and patterns, then modules and programs, and Chapter 4 gives each of these a dynamic counterpart.

3.1 Auxiliary definitions

Three syntactic analyses support the well-formedness rules: $\mathrm{assigns}(t)$, the set of variables assigned anywhere in t (without descending into nested function definitions, which have their own scope, so that $\mathrm{assigns}(t) = \mathrm{dom}(\Delta)$ when $t : \mathsf{Assigns}\ \Delta$); $\mathrm{binds}(p)$, the set of variables introduced by a pattern p, and likewise $\mathrm{binds}(\vec{g})$ by the generators of a qualifier sequence; and $\mathrm{captures}(t)$, the set of variables from the enclosing scope that are captured by lambdas and nested function definitions within t, and likewise $\mathrm{captures}(\vec{g})$ by lambdas within a qualifier sequence, with companion $\mathrm{captures}_{\mathsf{e}}(e)$ on expressions. All are defined in §A.1.

3.2 Statements, expressions and patterns

Every well-formed statement either *definitely returns*, or *definitely assigns* a context Δ, which may be empty. The base cases are that `return e` definitely returns, and `x = e` definitely assigns $\{\mathtt{x} : \mathsf{tt}\}$. *Result types* R classify these outcomes: R is either $\mathsf{Returns}$ (definitely returns) or $\mathsf{Assigns}\ \Delta$ (definitely assigns Δ). Statement well-formedness $\Gamma \vdash t : R$ is given in Figure 3.1, expression well-formedness $\Gamma \vdash e$ in Figure 3.2, and pattern well-formedness in Figure 3.4.

Context operations $\otimes$ and $\oplus$ lift to a right semiring over result types R, with $\mathsf{Assigns}\ \emptyset$ as left and right unit for $\otimes$ and $\mathsf{Returns}$ as unit for $\oplus$ and zero for $\otimes$.

Definition 3.1 (Override and merge on result types)**.**

$$\begin{aligned} \mathsf{Assigns}\ \Delta \oplus \mathsf{Assigns}\ \Delta' &= \mathsf{Assigns}\ (\Delta \oplus \Delta') \\ R \oplus \mathsf{Returns} = \mathsf{Returns} \oplus R &= R \end{aligned}$$

$$\begin{aligned} \mathsf{Assigns}\ \Delta \otimes \mathsf{Assigns}\ \Delta' &= \mathsf{Assigns}\ (\Delta \otimes \Delta') \\ R \otimes \mathsf{Returns} = \mathsf{Returns} \otimes R &= \mathsf{Returns} \end{aligned}$$

The $\mathsf{Returns} \otimes R$ case never arises because the well-formedness judgement excludes unreachable statements.

A sequence $s\ s'$ is well-formed (seq) when s definitely assigns some Δ, s' is well-formed under $\Gamma \otimes \Delta$, and no variable captured by s is assigned in s'. A conditional combines its branches with $\oplus$, merging in $\mathsf{Assigns}\ \emptyset$ when there is no `else` branch. A mutual region (def) checks each body in a context in

which the region's function names and the body's parameters are definitely assigned and the body's other assigned variables are at ff; the function names must be distinct. A `match` (match, match-partial) requires its case patterns to be well-formed as a list, checks each case body in a context in which the variables bound by its pattern are definitely assigned, and merges the cases as a conditional merges its branches, merging in Assigns $\emptyset$ unless the last case is a catch-all, i.e. a variable or wildcard. A class declaration (class, class-extend) assigns a class entry C to its name, which must not already denote a class; its fields must be distinct and, for an extension, disjoint from those of the base class.

A variable is a well-formed expression when it is definitely assigned (var); a lambda checks its body with its parameters definitely assigned, and a comprehension checks its body with the variables bound by its generators definitely assigned (qual-generator). A lambda within a comprehension may not capture a generator variable, a side condition of list-comp, dict-comp and qual-generator, because Python mutates the variable on each iteration and the lambda would see its final value. Two rules resolve a qualified name through the context to an entry θ (Figure 3.2a). A module attribute reference $e.x$ is well-formed when, read as a qualified name, it resolves to a definitely assigned member (attr-module). A constructor call requires its name to resolve to a class and its arguments to cover the class's fields exactly, through field-map (constr). A variable bound to a module or class entry is not an expression on its own, and occurs only as the head of a qualified name.

Pattern subsumption $\Gamma \vdash p \sqsubseteq p'$ (Figure 3.3) is a preorder on patterns under which every value matched by p is also matched by p'. A pattern list is well-formed (Figure 3.4) when each pattern is linear (no pattern variable occurs more than once), each constructor pattern names a class and its sub-patterns cover the class's fields exactly (pat-constr), and no earlier pattern subsumes a later one (cases-cons), so that every case of a `match` is reachable. Exhaustiveness is a separate test; without a type system to bound the values a match may receive, it degenerates to whether the last case is a catch-all.

3.3 Modules and programs

Static loading checks a module's body and computes its signature Γ, the judgement $\vdash_{\mathsf{load}} q : \Gamma$ (Figure 3.7); it mirrors loading at run time (Chapter 4). Both import forms statically load their target, together with its ancestors (Figure 3.6). A plain import also records each of these modules as a member of its parent (the static loads-as judgement), so that the binding created for the root of the target gives attribute access along the whole chain. A from-import also loads any imported name that denotes a submodule. An ancestor of the target is exempt from loading first if it is the importing module or one of its ancestors, since at runtime it may still be loading. The exemption applies only to from-imports, because for a plain import the only ancestor that could never load first is the importing module itself, and an import naming a proper descendant of the importing module is excluded outright. Loading a module may thus require other modules to load first; the rules are interpreted inductively, so a program with an import cycle is ill-formed, because a module in a cycle could finish loading only after itself. Program well-formedness (Figure 3.7) requires the main module to be well-formed and hence, recursively, every module the program can load; a module that is never imported is not checked. The rule module checks a module body under the context of $\mathcal{P}$(`builtins`), overridden by the imported context, so the members of `builtins` are in scope unqualified.

The signature Γ that static loading produces (Figure 3.7) comprises the classes and variables a module defines, together with its intrinsic `__name__` and a stub for each submodule, never the names it imports: following PEP 484[1], an imported name is a private dependency, not re-exported as part of the interface.

[1] https://peps.python.org/pep-0484/

$\Gamma \vdash t : R$

pass

$$\frac{}{\Gamma \vdash \texttt{pass} : \mathsf{Assigns}\ \emptyset}$$

assign

$$\frac{\Gamma \vdash e \qquad x \notin \mathrm{captures}_{\mathrm{e}}(e)}{\Gamma \vdash x\pi = e : \mathsf{Assigns}\ \{x : \mathsf{tt}\}}$$

expr-stmt

$$\frac{\Gamma \vdash e}{\Gamma \vdash e : \mathsf{Assigns}\ \emptyset}$$

assert

$$\frac{\Gamma \vdash e}{\Gamma \vdash \texttt{assert}\ e : \mathsf{Assigns}\ \emptyset}$$

assert-msg

$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash \texttt{assert}\ e, e' : \mathsf{Assigns}\ \emptyset}$$

return-none

$$\frac{}{\Gamma \vdash \texttt{return} : \mathsf{Returns}}$$

return

$$\frac{\Gamma \vdash e}{\Gamma \vdash \texttt{return}\ e : \mathsf{Returns}}$$

if

$$\frac{\Gamma \vdash e_i \quad (\forall i) \qquad \Gamma \vdash s_i : R_i \quad (\forall i)}{\Gamma \vdash \texttt{if}\ e_1 :\ s_1\ \texttt{elif}\ e_2 :\ s_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ s_{n+1} : R_1 \oplus \ldots \oplus R_{n+1} \oplus \mathsf{Assigns}\ \emptyset}$$

if-else

$$\frac{\Gamma \vdash e_i \quad (\forall i) \qquad \Gamma \vdash s_i : R_i \quad (\forall i)}{\Gamma \vdash \texttt{if}\ e_1 :\ s_1\ \texttt{elif}\ e_2 :\ s_2\ \ldots\ \texttt{elif}\ e_{n+1} :\ s_{n+1}\ \texttt{else} :\ s_{n+2} : R_1 \oplus \ldots \oplus R_{n+2}}$$

def

$$\frac{\Gamma \otimes \mathsf{tt}(\vec{f} +\!\!+ \vec{x}_i) \otimes \mathsf{ff}(\mathrm{assigns}(s_i) \setminus \vec{x}_i) \vdash s_i : R_i \quad (\forall i) \qquad \vec{f}\ \mathrm{distinct}}{\Gamma \vdash \texttt{def}\ f_1(\vec{x}_1\ \vec{\pi}_1)\ \omega_1 :\ s_1 \cdot \ldots \cdot \texttt{def}\ f_{n+1}(\vec{x}_{n+1}\ \vec{\pi}_{n+1})\ \omega_{n+1} :\ s_{n+1} : \mathsf{Assigns}\ \mathsf{tt}(\vec{f})}$$

match

$$\frac{\Gamma \vdash e \qquad \Gamma \otimes \mathsf{tt}(\mathrm{binds}(p_i)) \vdash s_i : R_i \quad (\forall i) \qquad S_i = \mathsf{Assigns}\ \mathsf{tt}(\mathrm{binds}(p_i)) \otimes R_i \quad (\forall i) \qquad \Gamma \vdash \vec{p} \qquad p_{n+1} \in \{x, _\}}{\Gamma \vdash \texttt{match}\ e\texttt{:}\ \texttt{case}\ p_1\texttt{:}\ s_1\ \ldots\ \texttt{case}\ p_{n+1}\texttt{:}\ s_{n+1} : S_1 \oplus \ldots \oplus S_{n+1}}$$

match-partial

$$\frac{\Gamma \vdash e \qquad \Gamma \otimes \mathsf{tt}(\mathrm{binds}(p_i)) \vdash s_i : R_i \quad (\forall i) \qquad S_i = \mathsf{Assigns}\ \mathsf{tt}(\mathrm{binds}(p_i)) \otimes R_i \quad (\forall i) \qquad \Gamma \vdash \vec{p} \qquad p_{n+1} \notin \{x, _\}}{\Gamma \vdash \texttt{match}\ e\texttt{:}\ \texttt{case}\ p_1\texttt{:}\ s_1\ \ldots\ \texttt{case}\ p_{n+1}\texttt{:}\ s_{n+1} : S_1 \oplus \ldots \oplus S_{n+1} \oplus \mathsf{Assigns}\ \emptyset}$$

seq

$$\frac{\Gamma \vdash t : \mathsf{Assigns}\ \Delta \qquad \Gamma \otimes \Delta \vdash t' : R \qquad \mathrm{captures}(t) \cap \mathrm{assigns}(t') = \emptyset}{\Gamma \vdash t\ t' : \mathsf{Assigns}\ \Delta \otimes R}$$

class

$$\frac{\nexists C'.\ c\texttt{:}\ C' \in \Gamma \qquad \text{own-fields}(\phi) = \vec{x}\ \mathrm{distinct} \qquad C = \mathsf{Cls}_\Gamma(q.c, \vec{x}, \bot)}{\Gamma \vdash \texttt{@dataclass class}\ c :\ \phi : \mathsf{Assigns}\ \{c\texttt{:}\ C\}}$$

class-extend

$$\frac{\nexists C'.\ c\texttt{:}\ C' \in \Gamma \qquad c'\texttt{:}\ \mathsf{Cls}_{\Gamma'}(q.c', \vec{y}, c^\dagger) \in \Gamma \qquad \text{own-fields}(\phi) = \vec{x}\ \mathrm{distinct} \qquad \vec{x} \cap \mathrm{fields}(\Gamma(c')) = \emptyset \qquad C = \mathsf{Cls}_\Gamma(q.c, \vec{x}, c')}{\Gamma \vdash \texttt{@dataclass class}\ c(c') :\ \phi : \mathsf{Assigns}\ \{c\texttt{:}\ C\}}$$

Figure 3.1: Statement t is well-formed in Γ with result type R

$\Gamma \vdash q : \theta$

simple-var
$$\frac{x : \mathsf{tt} \in \Gamma}{\Gamma \vdash x : \mathsf{tt}}$$

simple-module
$$\frac{x : \mathsf{Mod}(q, \Gamma') \in \Gamma}{\Gamma \vdash x : \mathsf{Mod}(q, \Gamma')}$$

simple-class
$$\frac{c : C \in \Gamma}{\Gamma \vdash c : C}$$

qualified
$$\frac{\Gamma \vdash q : \mathsf{Mod}(q', \Gamma') \qquad \Gamma' \vdash x : \theta}{\Gamma \vdash q.x : \theta}$$

(a) Qualified name q resolves to context entry θ under Γ

$\Gamma \vdash e$

var
$$\frac{x : \mathsf{tt} \in \Gamma}{\Gamma \vdash x}$$

literal
$$\frac{}{\Gamma \vdash \ell}$$

lambda
$$\frac{\Gamma \otimes \mathsf{tt}(\vec{x}) \vdash e}{\Gamma \vdash \texttt{lambda}\ \vec{x} :\ e}$$

call
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e_i \quad (\forall i)}{\Gamma \vdash e(e_1, \ldots, e_n)}$$

constr
$$\frac{\Gamma \vdash q : C \qquad \Gamma \vdash \text{field-map}(C, \vec{e})(x) \quad (\forall x \in \text{fields}(C))}{\Gamma \vdash q(\vec{e})}$$

attr-module
$$\frac{\Gamma \vdash e.x : \mathsf{tt}}{\Gamma \vdash e.x}$$

attr-object
$$\frac{\Gamma \vdash e}{\Gamma \vdash e.x}$$

dict
$$\frac{\Gamma \vdash e_i \quad (\forall i) \qquad \Gamma \vdash e'_i \quad (\forall i)}{\Gamma \vdash \{e_1 : e'_1, \ldots, e_n : e'_n\}}$$

list
$$\frac{\Gamma \vdash e_i \quad (\forall i)}{\Gamma \vdash [e_1, \ldots, e_n]}$$

tuple
$$\frac{\Gamma \vdash e_i \quad (\forall i)}{\Gamma \vdash (e_1, \ldots, e_n)}$$

subscript
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash e[e']}$$

binop
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash e \odot e'}$$

unop
$$\frac{\Gamma \vdash e}{\Gamma \vdash \ominus e}$$

and
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash e\ \texttt{and}\ e'}$$

or
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash e\ \texttt{or}\ e'}$$

cond
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e_1 \qquad \Gamma \vdash e_2}{\Gamma \vdash e_1\ \texttt{if}\ e\ \texttt{else}\ e_2}$$

list-comp
$$\frac{\Gamma \vdash \vec{g} : \Delta \qquad \Gamma \otimes \Delta \vdash e \qquad \text{captures}_e(e) \cap \text{dom}(\Delta) = \emptyset}{\Gamma \vdash [\ e\ g_1\ \ldots\ g_n]}$$

dict-comp
$$\frac{\Gamma \vdash \vec{g} : \Delta \qquad \Gamma \otimes \Delta \vdash e \qquad \Gamma \otimes \Delta \vdash e' \qquad (\text{captures}_e(e) \cup \text{captures}_e(e')) \cap \text{dom}(\Delta) = \emptyset}{\Gamma \vdash \{e : e'\ g_1\ \ldots\ g_n\}}$$

(b) Expression e is well-formed under Γ

$\Gamma \vdash \vec{g} : \Delta$

quals-nil
$$\frac{}{\Gamma \vdash \varepsilon : \emptyset}$$

qual-guard
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash \vec{g} : \Delta}{\Gamma \vdash \texttt{if}\ e \cdot \vec{g} : \Delta}$$

qual-generator
$$\frac{\Gamma \vdash e \qquad \Gamma \otimes \{x : \mathsf{tt}\} \vdash \vec{g} : \Delta \qquad x \notin \text{captures}(\vec{g})}{\Gamma \vdash \texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g} : \{x : \mathsf{tt}\} \otimes \Delta}$$

(c) Comprehension qualifiers $\vec{g}$ bind Δ under Γ

Figure 3.2: Expression well-formedness

$\Gamma \vdash p \sqsubseteq p'$

sub-lit
$$\frac{}{\Gamma \vdash \ell \sqsubseteq \ell}$$

sub-neg-lit
$$\frac{}{\Gamma \vdash -n \sqsubseteq -n}$$

sub-var
$$\frac{}{\Gamma \vdash p \sqsubseteq x}$$

sub-wild
$$\frac{}{\Gamma \vdash p \sqsubseteq _}$$

sub-list-list
$$\frac{\Gamma \vdash p_i \sqsubseteq q_i \quad (\forall i)}{\Gamma \vdash [p_1, \ldots, p_n] \sqsubseteq [q_1, \ldots, q_n]}$$

sub-tuple-tuple
$$\frac{\Gamma \vdash p_i \sqsubseteq q_i \quad (\forall i)}{\Gamma \vdash (p_1, \ldots, p_n) \sqsubseteq (q_1, \ldots, q_n)}$$

sub-dict-dict
$$\frac{\{u_1, \ldots, u_m\} \subseteq \{w_1, \ldots, w_n\} \qquad \Gamma \vdash p_i \sqsubseteq q_j \quad (\forall i, j.\ w_i = u_j)}{\Gamma \vdash \{w_1 : p_1, \ldots, w_n : p_n\} \sqsubseteq \{u_1 : q_1, \ldots, u_m : q_m\}}$$

sub-constr
$$\frac{\Gamma \vdash q : C \qquad \Gamma \vdash q' : C' \qquad C' \in \text{ancestors}(C) \qquad \Gamma \vdash \text{field-map}(C, \vec{p})(x) \sqsubseteq \text{field-map}(C', \vec{p}\,')(x) \quad (\forall x \in \text{fields}(C'))}{\Gamma \vdash q(\vec{p}) \sqsubseteq q'(\vec{p}\,')}$$

sub-as-l
$$\frac{\Gamma \vdash p \sqsubseteq q}{\Gamma \vdash p \texttt{ as } x \sqsubseteq q}$$

sub-as-r
$$\frac{\Gamma \vdash p \sqsubseteq q}{\Gamma \vdash p \sqsubseteq q \texttt{ as } y}$$

Figure 3.3: Pattern p is subsumed by pattern p' under Γ

$\Gamma \vdash p_1 \cdot \ldots \cdot p_{n+1}$

pat-lit
$$\frac{}{\Gamma \vdash \ell}$$

pat-neg-lit
$$\frac{}{\Gamma \vdash -n}$$

pat-var
$$\frac{}{\Gamma \vdash x}$$

pat-wild
$$\frac{}{\Gamma \vdash _}$$

pat-list
$$\frac{\Gamma \vdash p_i \quad (\forall i) \qquad \text{variables in } \vec{p} \text{ distinct}}{\Gamma \vdash [\vec{p}]}$$

pat-tuple
$$\frac{\Gamma \vdash p_i \quad (\forall i) \qquad \text{variables in } \vec{p} \text{ distinct}}{\Gamma \vdash (\vec{p})}$$

pat-as
$$\frac{\Gamma \vdash p \qquad x \notin \text{binds}(p)}{\Gamma \vdash p \texttt{ as } x}$$

pat-dict
$$\frac{\Gamma \vdash p_i \quad (\forall i) \qquad w_1, \ldots, w_n \text{ distinct} \qquad \text{variables in } \vec{p} \text{ distinct}}{\Gamma \vdash \{w_1 : p_1, \ldots, w_n : p_n\}}$$

pat-constr
$$\frac{\Gamma \vdash q : C \qquad \Gamma \vdash \text{field-map}(C, \vec{p})(x) \quad (\forall x \in \text{fields}(C)) \qquad \text{variables in } \vec{p} \text{ distinct}}{\Gamma \vdash q(\vec{p})}$$

cases-cons
$$\frac{\Gamma \vdash p \qquad \Gamma \vdash p' \cdot \vec{p}\,' \qquad \Gamma \vdash p_i \not\sqsubseteq p \quad (\forall i)}{\Gamma \vdash p \cdot p' \cdot \vec{p}\,'}$$

Figure 3.4: Case patterns are well-formed under Γ

Γ, x imports θ

$$\text{imp-member}\ \frac{\Gamma \vdash x : \theta}{\Gamma, x \text{ imports } \theta} \qquad \text{imp-submodule}\ \frac{x : \mathsf{Mod}(q) \in \Gamma \qquad \vdash_{\mathsf{load}} q : \Delta}{\Gamma, x \text{ imports } \mathsf{Mod}(q, \Delta)}$$

Figure 3.5: Name x imported from module signature Γ has entry θ

$\vdash \vec{\imath} : \Gamma$

$$\text{imports-nil}\ \frac{}{\vdash \varepsilon : \emptyset} \qquad \text{import}\ \frac{q' = x \cdot \vec{x} \qquad q \not\prec q' \qquad \vdash_{\mathsf{load}} q' : \Delta \qquad q', \mathsf{Mod}(q', \Delta) \text{ loads-as } \theta \qquad \vdash \vec{\imath} : \Gamma}{\vdash \texttt{import}\ q' \cdot \vec{\imath} : \{x : \theta\} \lhd \Gamma}$$

$$\text{from-import}\ \frac{\vdash_{\mathsf{load}} q' : \Delta \qquad \vdash_{\mathsf{load}} q^\dagger : \Delta_{q^\dagger} \quad (\forall q^\dagger \prec q',\ q^\dagger \not\preceq q) \qquad \Delta, x_i \text{ imports } \theta_i \quad (\forall i) \qquad \vdash \vec{\imath} : \Gamma}{\vdash \texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} : \{x_1 : \theta_1, \ldots, x_{n+1} : \theta_{n+1}\} \lhd \Gamma}$$

(a) Import prefix $\vec{\imath}$ of module q produces bindings Γ

q, θ loads-as θ'

$$\text{loads-as-simple}\ \frac{}{x, \theta \text{ loads-as } \theta} \qquad \text{loads-as-qualified}\ \frac{\vdash_{\mathsf{load}} q : \Gamma' \qquad q, \mathsf{Mod}(q, \Gamma' \lhd \{x : \theta\}) \text{ loads-as } \theta'}{q.x, \theta \text{ loads-as } \theta'}$$

(b) Reference θ for module q loads as the reference θ' for the root of q, statically loading the ancestors of q

Figure 3.6: Import well-formedness

$\vdash_{\mathsf{load}} q : \Gamma$

$$\text{predefined}\ \frac{\mathcal{P}(q) = (\rho, \Gamma)}{\vdash_{\mathsf{load}} q : \Gamma}$$

$$\text{module}\ \frac{\mathcal{M}(q) = \vec{\imath}\ t \qquad \vdash \vec{\imath} : \Gamma \qquad \mathcal{P}(\texttt{builtins}) = (\rho, \Gamma') \qquad \Gamma' \otimes \Gamma \vdash t : \mathsf{Assigns}\ \Delta \qquad (\mathrm{dom}(\Gamma) \cup \mathrm{dom}(\Delta)) \cap \mathrm{dom}(\mathrm{submods}(q)) = \emptyset}{\vdash_{\mathsf{load}} q : \mathrm{submods}(q) \otimes \Delta}$$

$\vdash \mathcal{M}$

$$\text{program}\ \frac{\vdash_{\mathsf{load}} \texttt{__main__} : \Gamma}{\vdash \mathcal{M}}$$

Figure 3.7: Module well-formedness

Submodule stubs and the module's own bindings are disjoint, because the module rules exclude a module that binds a name that is also the name of a submodule.

4 Operational semantics

4.1 Values and environments

Values, environments, and evaluation results are given in Figure 4.1. For a statement s, the result $\mathsf{assigns}\ \rho$ represents *normal completion* of s, with accumulated environment ρ; the result $\mathsf{returns}\ v$ represents *abrupt completion* of s, with return value v; and the result $\mathsf{aborts}\ \kappa$ represents abrupt completion of the whole run, with termination kind κ. Composing statements sequentially composes their results according to the following partial monoid, analogous to sequential composition of result types:

$$\begin{aligned}\mathsf{assigns}\ \rho \otimes \mathsf{assigns}\ \rho' &= \mathsf{assigns}\ (\rho \otimes \rho')\\ \mathsf{assigns}\ \rho \otimes \mathsf{returns}\ v &= \mathsf{returns}\ v\\ \mathsf{assigns}\ \rho \otimes \mathsf{aborts}\ \kappa &= \mathsf{aborts}\ \kappa\end{aligned}$$

with $\mathsf{assigns}\ \emptyset$ as left and right unit.

A *termination kind* κ records how a run ends; all but $\mathsf{SystemExit}(n)$ are named after the exceptions the same program raises under Python in the same circumstances. A primitive $\mathsf{Prim}(f)$ is a predefined function whose meaning f is a partial map from arguments to outcomes (Figure 2.7). Calling $\mathsf{Prim}(f)$ with arguments $\vec{u}$ has outcome $f(\vec{u})$, and no derivation where $f(\vec{u})$ is undefined.

The rules are given in a monadic metalanguage over outcomes (Figure 4.1), in which $\mathsf{let}\ v \leftarrow o\ \mathsf{in}\ \ldots$ binds the value of an outcome; for example, eval-binop concludes $\mathsf{let}\ v' \leftarrow o\ \mathsf{in}\ \hat{\odot}(v, v')$, which is $\hat{\odot}(v, v')$ where o is $\mathsf{val}\ v'$, and $\mathsf{aborts}\ \kappa$ where o is $\mathsf{aborts}\ \kappa$. A final premise needs no propagation rule; earlier premises have one each, recording which subexpressions succeeded before the failing one was reached (Figure 4.9). The statement, sequence and module judgements use outcomes of the same form, carrying a result, a sequence of values or an environment in place of a value.

4.2 Auxiliary definitions

The operators and the other auxiliary functions the evaluation rules use are defined in §A.3, and module and class resolution in §A.2.

4.3 Pattern matching

A `match` statement takes the first case whose pattern matches the value, running its body under the bindings the pattern produces, and does nothing if no case matches; the auxiliary function dispatch (§A.3) selects the matching case. Whether a pattern matches is determined by the pattern-matching judgement $\rho, p, v \rightsquigarrow m$ (Figure 4.2). A *match result* m is either $\mathsf{match}\ \rho'$, a match, where ρ' binds the pattern's variables to the values they matched, or $\mathsf{no\text{-}match}$, a failure to match. Where a pattern and a value are not comparable, as a list pattern against a tuple, there is no derivation and the run has no result. Sub-results compose with $\otimes$, matching sequentially:

$$\mathsf{match}\ \rho \otimes \mathsf{match}\ \rho' = \mathsf{match}\ (\rho \uplus \rho')$$

Values			
v	$::=$	ℓ	(Literal)
	$\mid$	$[v_1, \ldots, v_n]$	(List)
	$\mid$	$(v_1, \ldots, v_n)$	(Tuple)
	$\mid$	$\{\delta\}$	(Dict)
	$\mid$	$\mathsf{Lam}_\Gamma(\rho, \vec{x}, e)$	(LambdaClosure)
	$\mid$	$\mathsf{Def}_\Gamma(\rho, \vec{d}, i)$	(DefClosure)
	$\mid$	$\mathsf{Obj}_\Gamma(C, \rho)$	(Object)
	$\mid$	$\mathsf{Mod}(q)$	(ModuleStub)
	$\mid$	$\mathsf{Mod}(q, \rho)$	(LoadedModule)
	$\mid$	$\mathsf{Cls}_\Gamma(q.c, \vec{x}, c')$	(Class)
	$\mid$	$\mathsf{Prim}(f)$	(Primitive)
Environments			
ρ	$::=$	$\{x_1 : v_1, \ldots, x_n : v_n\}$	
Dictionary entries			
δ	$::=$	$w_1 : v_1 \cdot \ldots \cdot w_n : v_n$	
Results			
r	$::=$	assigns ρ	(Assigns)
	$\mid$	returns v	(Returns)
	$\mid$	aborts κ	(Aborts)
Outcomes			
o	$::=$	val v	(Val)
	$\mid$	aborts κ	(Aborts)
Match results			
m	$::=$	match ρ	(Match)
	$\mid$	no-match	(NoMatch)
Termination kinds			
κ	$::=$	TypeError $\mid$ IndexError $\mid$ KeyError $\mid$ ZeroDivisionError	
	$\mid$	AssertionError $\mid$ AssertionError(w) $\mid$ AttributeError $\mid$ SystemExit(n)	

Figure 4.1: Values, environments and evaluation results

$$m \otimes \text{no-match} = \text{no-match} \otimes m = \text{no-match}$$

with match $\emptyset$ as left and right unit. A constructor pattern matches any object whose class has the pattern's class among its ancestors, and its sub-patterns bind the fields of the pattern's class. An object records its class entry, whose qualified name is the identity of the class: since a module declares each class name at most once, the entry comparison in eval-pat-constr amounts to a comparison of qualified names.

4.4 Statements and expressions

Statement evaluation $\rho, t \Rightarrow r$ is given in Figures 4.3 and 4.4. Function definitions delimit the scope of sequential-composition effects, so as black boxes functions are always pure. The auxiliary function outcome (§A.3) reads a statement's result as the outcome of calling it, val `None` where the statement

$\rho, p, v \rightsquigarrow m$

eval-pat-lit
$$\frac{\mathrm{eq}(\ell, v) = \texttt{True}}{\rho, \ell, v \rightsquigarrow \text{match } \emptyset}$$

eval-pat-lit-no
$$\frac{\mathrm{eq}(\ell, v) = \texttt{False}}{\rho, \ell, v \rightsquigarrow \text{no-match}}$$

eval-pat-neg-lit
$$\frac{\mathrm{eq}(-n, v) = \texttt{True}}{\rho, -n, v \rightsquigarrow \text{match } \emptyset}$$

eval-pat-neg-lit-no
$$\frac{\mathrm{eq}(-n, v) = \texttt{False}}{\rho, -n, v \rightsquigarrow \text{no-match}}$$

eval-pat-var
$$\frac{}{\rho, x, v \rightsquigarrow \text{match } \{x : v\}}$$

eval-pat-wild
$$\frac{}{\rho, _, v \rightsquigarrow \text{match } \emptyset}$$

eval-pat-list
$$\frac{\rho, p_i, v_i \rightsquigarrow m_i \quad (\forall i)}{\rho, [p_1, \ldots, p_n], [v_1, \ldots, v_n] \rightsquigarrow m_1 \otimes \ldots \otimes m_n}$$

eval-pat-list-no
$$\frac{v \text{ neither list nor tuple, or list of length} \neq n}{\rho, [p_1, \ldots, p_n], v \rightsquigarrow \text{no-match}}$$

eval-pat-tuple
$$\frac{\rho, p_i, v_i \rightsquigarrow m_i \quad (\forall i)}{\rho, (p_1, \ldots, p_n), (v_1, \ldots, v_n) \rightsquigarrow m_1 \otimes \ldots \otimes m_n}$$

eval-pat-tuple-no
$$\frac{v \text{ neither list nor tuple, or tuple of length} \neq n}{\rho, (p_1, \ldots, p_n), v \rightsquigarrow \text{no-match}}$$

eval-pat-dict
$$\frac{w_i : v_i \in \delta \quad (\forall i) \qquad \rho, p_i, v_i \rightsquigarrow m_i \quad (\forall i)}{\rho, \{w_1 : p_1, \ldots, w_n : p_n\}, \delta \rightsquigarrow m_1 \otimes \ldots \otimes m_n}$$

eval-pat-dict-no
$$\frac{v \text{ not dictionary, or } \delta \text{ with } w_i \notin \mathrm{dom}(\delta) \text{ for some } i}{\rho, \{w_1 : p_1, \ldots, w_n : p_n\}, v \rightsquigarrow \text{no-match}}$$

eval-pat-as
$$\frac{\rho, p, v \rightsquigarrow m}{\rho, p \texttt{ as } x, v \rightsquigarrow m \otimes \text{match } \{x : v\}}$$

eval-pat-constr
$$\frac{\text{resolve-class}(\rho, q) = C \qquad C \in \text{ancestors}(C') \qquad \rho, \text{field-map}(C, \vec{p})(x), \rho'(x) \rightsquigarrow m_x \quad (\forall x \in \text{fields}(C))}{\rho, q(\vec{p}), \mathsf{Obj}_{\Gamma'}(C', \rho') \rightsquigarrow \bigotimes_{x \in \text{fields}(C)} m_x}$$

eval-pat-constr-no
$$\frac{\text{resolve-class}(\rho, q) = C \qquad v \text{ not object, or } \mathsf{Obj}_{\Gamma'}(C', \rho') \text{ with } C \notin \text{ancestors}(C')}{\rho, q(\vec{p}), v \rightsquigarrow \text{no-match}}$$

Figure 4.2: Pattern p matches value v under environment ρ, with result m

completes without returning.

Expression evaluation $\rho, e \Rightarrow o$ (Figures 4.5 and 4.6) yields an outcome, either val v or aborts κ (Figure 4.1). The operators `and` and `or` evaluate their second operand only when the first leaves the result open, and a conditional expression requires its condition to be `True` or `False`, as eval-if does. A comprehension evaluates its body once for each binding its qualifiers produce, and concatenates the results; a generator draws its bindings from the elements of a value, given by the auxiliary function iter (§A.3): the elements of a list, tuple or string, and the keys of a dictionary, as in Python. Subscripting, given by the auxiliary function getitem (§A.3), indexes a list, tuple or string by an integer, counting from the end when negative, and a dictionary by a string key, aborting with IndexError, KeyError or TypeError where Python raises the exception of that name; a dictionary subscripted by a value other than a string is undefined. Likewise an attribute reference on a value other than a module or object has no derivation: the attributes of built-in values are undefined.

$\rho, t \Rightarrow r$

eval-pass

$$\frac{}{\rho, \texttt{pass} \Rightarrow \text{assigns } \emptyset}$$

eval-assign

$$\frac{\rho, e \Rightarrow o}{\rho, x\pi = e \Rightarrow \text{let } v \leftarrow o \text{ in assigns } \{x : v\}}$$

eval-expr-stmt

$$\frac{\rho, e \Rightarrow o}{\rho, e \Rightarrow \text{let } v \leftarrow o \text{ in assigns } \emptyset}$$

eval-return

$$\frac{\rho, e \Rightarrow o}{\rho, \texttt{return } e \Rightarrow \text{let } v \leftarrow o \text{ in returns } v}$$

eval-return-none

$$\frac{}{\rho, \texttt{return} \Rightarrow \text{returns } \texttt{None}}$$

eval-def

$$\frac{d_i = \texttt{def } f_i(\vec{x}_i\, \vec{\pi}_i)\, \omega_i : s_i \quad (\forall i) \qquad v_i = \text{Def}_\Gamma(\rho, \vec{d}, i) \quad (\forall i)}{\rho, \vec{d} \Rightarrow \text{assigns } \{\vec{f} : \vec{v}\}}$$

eval-assert

$$\frac{\rho, e \Rightarrow \text{val } \texttt{True}}{\rho, \texttt{assert } e \Rightarrow \text{assigns } \emptyset}$$

eval-assert-msg

$$\frac{\rho, e \Rightarrow \text{val } \texttt{True}}{\rho, \texttt{assert } e, e' \Rightarrow \text{assigns } \emptyset}$$

eval-assert-false

$$\frac{\rho, e \Rightarrow \text{val } \texttt{False}}{\rho, \texttt{assert } e \Rightarrow \text{aborts AssertionError}}$$

eval-assert-msg-false

$$\frac{\rho, e \Rightarrow \text{val } \texttt{False} \qquad \rho, e' \Rightarrow o}{\rho, \texttt{assert } e, e' \Rightarrow \text{let } w \leftarrow o \text{ in aborts AssertionError}(w)}$$

eval-if

$$\frac{\rho, e_j \Rightarrow \text{val } \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \text{val } \texttt{True} \qquad \rho, s_i \Rightarrow r}{\rho, \texttt{if } e_1 : s_1 \texttt{ elif } e_2 : s_2 \ \ldots \ \texttt{elif } e_{n+1} : s_{n+1} \Rightarrow r}$$

eval-if-none

$$\frac{\rho, e_j \Rightarrow \text{val } \texttt{False} \quad (\forall j \in \{1, \ldots, n+1\})}{\rho, \texttt{if } e_1 : s_1 \texttt{ elif } e_2 : s_2 \ \ldots \ \texttt{elif } e_{n+1} : s_{n+1} \Rightarrow \text{assigns } \emptyset}$$

eval-if-else

$$\frac{\rho, e_j \Rightarrow \text{val } \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \text{val } \texttt{True} \qquad \rho, s_i \Rightarrow r}{\rho, \texttt{if } e_1 : s_1 \texttt{ elif } e_2 : s_2 \ \ldots \ \texttt{elif } e_{n+1} : s_{n+1} \texttt{ else : } s_{n+2} \Rightarrow r}$$

eval-else

$$\frac{\rho, e_j \Rightarrow \text{val } \texttt{False} \quad (\forall j \in \{1, \ldots, n+1\}) \qquad \rho, s_{n+2} \Rightarrow r}{\rho, \texttt{if } e_1 : s_1 \texttt{ elif } e_2 : s_2 \ \ldots \ \texttt{elif } e_{n+1} : s_{n+1} \texttt{ else : } s_{n+2} \Rightarrow r}$$

eval-match

$$\frac{\rho, e \Rightarrow \text{val } v \qquad \text{dispatch}(\rho, v, \vec{p}, \vec{s}) = (\rho', s') \qquad \rho \otimes \rho', s' \Rightarrow r}{\rho, \texttt{match } e\texttt{: case } p_1\texttt{: } s_1 \ \ldots \ \texttt{case } p_{n+1}\texttt{: } s_{n+1} \Rightarrow \text{assigns } \rho' \otimes r}$$

eval-seq

$$\frac{\rho, t \Rightarrow \text{assigns } \rho' \qquad \rho \otimes \rho', t' \Rightarrow r}{\rho, t\ t' \Rightarrow \text{assigns } \rho' \otimes r}$$

eval-class

$$\frac{C = \text{Cls}_\Gamma(q.c, \text{own-fields}(\phi), \bot)}{\rho, \texttt{@dataclass class } c : \phi \Rightarrow \text{assigns } \{c : C\}}$$

eval-class-extend

$$\frac{C = \text{Cls}_\Gamma(q.c, \text{own-fields}(\phi), c')}{\rho, \texttt{@dataclass class } c(c') : \phi \Rightarrow \text{assigns } \{c : C\}}$$

Figure 4.3: Statement t evaluates to result r under environment $\rho : \Gamma$

$\rho, t \Rightarrow r$ (continued)

eval-assert-fail

$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, \texttt{assert } e \Rightarrow \text{aborts } \kappa}$$

eval-assert-msg-fail

$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, \texttt{assert } e, e' \Rightarrow \text{aborts } \kappa}$$

eval-if-fail

$$\frac{\rho, e_j \Rightarrow \text{val } \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \text{aborts } \kappa}{\rho, \texttt{if } e_1 : s_1 \texttt{ elif } e_2 : s_2 \ \ldots \ \texttt{elif } e_{n+1} : s_{n+1} \Rightarrow \text{aborts } \kappa}$$

eval-if-else-fail

$$\frac{\rho, e_j \Rightarrow \text{val } \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \text{aborts } \kappa}{\rho, \texttt{if } e_1 : s_1 \texttt{ elif } e_2 : s_2 \ \ldots \ \texttt{elif } e_{n+1} : s_{n+1} \texttt{ else } : s_{n+2} \Rightarrow \text{aborts } \kappa}$$

eval-match-fail

$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, \texttt{match } e\texttt{:} \texttt{ case } p_1\texttt{:} \ s_1 \ \ldots \ \texttt{case } p_{n+1}\texttt{:} \ s_{n+1} \Rightarrow \text{aborts } \kappa}$$

eval-seq-fail

$$\frac{\rho, t \Rightarrow \text{aborts } \kappa}{\rho, t \ t' \Rightarrow \text{aborts } \kappa}$$

Figure 4.4: Statement evaluation, continued (aborting cases)

4.5 Modules and programs

Module and program evaluation are given in Figure 4.13; import evaluation in Figure 4.12; importing names from a loaded module in Figure 4.11. Context extension (Figure 2.2) applies to environments as well, with the same equations on values in place of context entries, except that a loaded module value $\text{Mod}(q, \rho)$ carries an environment where the context entry $\text{Mod}(q, \Gamma)$ carries a context. *Loading*, the judgement $q \Rightarrow_{\text{load}} o$, evaluates the import prefix of q and then its statement, mirroring static loading $\vdash_{\text{load}} q : \Gamma$ (Figure 3.7); as with the static rules, the mutual recursion between loading and evaluation is grounded by program well-formedness, which rules out import cycles (§2.3). Loading takes a single module; an import loads the ancestors of its target from the root down, each before its descendant, as in the static rules, by the loads-as judgement for a plain import and as premises of the from-import rule.

Since imports precede statements, every module a statement can reference is loaded before the statement runs, and loading is deterministic, so the semantics needs no cache of loaded modules. The rule eval-module puts $\mathcal{P}(\texttt{builtins})$ under a module body's environment, mirroring module (§3.3). A program evaluates to a termination kind (Figure 4.1): $\text{SystemExit}(0)$ if the main module loads, and otherwise the kind it aborted with. $\text{SystemExit}(n)$ is a run that exited with status n. An implementation must agree on which kind a run yields, though how it reports one is not prescribed.

$\rho, e \Rightarrow o$

eval-var

$$\frac{x : v \in \rho}{\rho, x \Rightarrow \mathsf{val}\ v}$$

eval-literal

$$\frac{}{\rho, \ell \Rightarrow \mathsf{val}\ \ell}$$

eval-lambda

$$\frac{}{\rho, \texttt{lambda}\ x_1, \ldots, x_n : \ e \Rightarrow \mathsf{val}\ \mathsf{Lam}_\Gamma(\rho, \vec{x}, e)}$$

eval-unop

$$\frac{\rho, e \Rightarrow o}{\rho, \ominus\, e \Rightarrow \mathsf{let}\ v \leftarrow o\ \mathsf{in}\ \hat{\ominus}(v)}$$

eval-binop

$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \rho, e' \Rightarrow o}{\rho, e \odot e' \Rightarrow \mathsf{let}\ v' \leftarrow o\ \mathsf{in}\ \hat{\odot}(v, v')}$$

eval-list

$$\frac{\rho, e_1 \cdot \ldots \cdot e_n \Rightarrow \mathsf{val}\ (v_1 \cdot \ldots \cdot v_n)}{\rho, [e_1, \ldots, e_n] \Rightarrow \mathsf{val}\ [v_1, \ldots, v_n]}$$

eval-tuple

$$\frac{\rho, e_1 \cdot \ldots \cdot e_n \Rightarrow \mathsf{val}\ (v_1 \cdot \ldots \cdot v_n)}{\rho, (e_1, \ldots, e_n) \Rightarrow \mathsf{val}\ (v_1, \ldots, v_n)}$$

eval-dict

$$\frac{\rho, e_1 \cdot e_1' \cdot \ldots \cdot e_n \cdot e_n' \Rightarrow \mathsf{val}\ (w_1 \cdot v_1 \cdot \ldots \cdot w_n \cdot v_n)}{\rho, \{e_1 : e_1', \ldots, e_n : e_n'\} \Rightarrow \mathsf{val}\ \{\mathrm{entries}(\varepsilon, w_1 : v_1 \cdot \ldots \cdot w_n : v_n)\}}$$

eval-list-comp

$$\frac{\rho, \vec{g} \Rightarrow \mathsf{val}\ (\rho_1 \cdot \ldots \cdot \rho_m) \qquad \rho_i, e \Rightarrow \mathsf{val}\ v_i \quad (\forall i)}{\rho, [\ e\ g_1\ \ldots\ g_n] \Rightarrow \mathsf{val}\ [v_1, \ldots, v_m]}$$

eval-dict-comp

$$\frac{\rho, \vec{g} \Rightarrow \mathsf{val}\ (\rho_1 \cdot \ldots \cdot \rho_m) \qquad \rho_i, e \Rightarrow \mathsf{val}\ w_i \quad (\forall i) \qquad \rho_i, e' \Rightarrow \mathsf{val}\ v_i \quad (\forall i)}{\rho, \{e : e'\ g_1\ \ldots\ g_n\} \Rightarrow \mathsf{val}\ \{\mathrm{entries}(\varepsilon, w_1 : v_1 \cdot \ldots \cdot w_m : v_m)\}}$$

eval-attr-module

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \mathsf{Mod}(q', \rho') \qquad x : v \in \rho'}{\rho, e.x \Rightarrow \mathsf{val}\ v}$$

eval-attr-object

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \mathsf{Obj}_\Delta(C, \rho')}{\rho, e.x \Rightarrow \mathsf{val}\ \rho'(x)}$$

eval-constr

$$\frac{\rho, e_1 \cdot \ldots \cdot e_{n+m} \Rightarrow \mathsf{val}\ (v_1 \cdot \ldots \cdot v_{n+m}) \qquad \text{resolve-class}(\rho, q) = C}{\rho, q(e_1, \ldots, e_n, y_1{=}e_{n+1}, \ldots, y_m{=}e_{n+m}) \Rightarrow \mathsf{val}\ \mathsf{Obj}_\Gamma(C, \text{field-map}(C, (v_1, \ldots, v_n, y_1{=}v_{n+1}, \ldots, y_m{=}v_{n+m})))}$$

Figure 4.5: Expression e evaluates to outcome o under environment $\rho : \Gamma$

$\rho, e \Rightarrow o$ (continued)

eval-subscript

$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \rho, e' \Rightarrow o}{\rho, e[e'] \Rightarrow \mathsf{let}\ v' \leftarrow o\ \mathsf{in}\ \mathrm{getitem}(v, v')}$$

eval-call-lambda

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \mathsf{Lam}_\Delta(\rho', \vec{x}, e') \qquad \rho, \vec{e} \Rightarrow \mathsf{val}\ \vec{u} \qquad \rho' \otimes \{\vec{x} : \vec{u}\}, e' \Rightarrow o}{\rho, e(e_1, \ldots, e_n) \Rightarrow o}$$

eval-call-def

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \mathsf{Def}_\Delta(\rho', \vec{d}, j) \qquad d_i = \texttt{def}\ f_i(\vec{x}_i\ \vec{\pi}_i)\ \omega_i : s_i \quad (\forall i) \qquad v_i = \mathsf{Def}_\Delta(\rho', \vec{d}, i) \quad (\forall i) \qquad \rho, \vec{e} \Rightarrow \mathsf{val}\ \vec{u} \qquad \rho' \otimes \{\vec{f} : \vec{v}\} \otimes \{\vec{x}_j : \vec{u}\}, s_j \Rightarrow r}{\rho, e(e_1, \ldots, e_n) \Rightarrow \mathrm{outcome}(r)}$$

eval-call-prim

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \mathsf{Prim}(f) \qquad \rho, \vec{e} \Rightarrow \mathsf{val}\ \vec{u}}{\rho, e(e_1, \ldots, e_n) \Rightarrow f(\vec{u})}$$

eval-and-false

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{False}}{\rho, e\ \texttt{and}\ e' \Rightarrow \mathsf{val}\ \texttt{False}}$$

eval-and-true

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{True} \qquad \rho, e' \Rightarrow o}{\rho, e\ \texttt{and}\ e' \Rightarrow o}$$

eval-or-true

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{True}}{\rho, e\ \texttt{or}\ e' \Rightarrow \mathsf{val}\ \texttt{True}}$$

eval-or-false

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{False} \qquad \rho, e' \Rightarrow o}{\rho, e\ \texttt{or}\ e' \Rightarrow o}$$

eval-cond-true

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{True} \qquad \rho, e_1 \Rightarrow o}{\rho, e_1\ \texttt{if}\ e\ \texttt{else}\ e_2 \Rightarrow o}$$

eval-cond-false

$$\frac{\rho, e \Rightarrow \mathsf{val}\ \texttt{False} \qquad \rho, e_2 \Rightarrow o}{\rho, e_1\ \texttt{if}\ e\ \texttt{else}\ e_2 \Rightarrow o}$$

Figure 4.6: Expression evaluation, continued

$\rho, \vec{e} \Rightarrow o$

eval-exprs-nil

$$\frac{}{\rho, \varepsilon \Rightarrow \mathsf{val}\ \varepsilon}$$

eval-exprs-cons

$$\frac{\rho, e \Rightarrow \mathsf{val}\ v \qquad \rho, \vec{e} \Rightarrow o}{\rho, e \cdot \vec{e} \Rightarrow \mathsf{let}\ \vec{v} \leftarrow o\ \mathsf{in}\ \mathsf{val}\ (v \cdot \vec{v})}$$

eval-exprs-fail

$$\frac{\rho, e \Rightarrow \mathsf{aborts}\ \kappa}{\rho, e \cdot \vec{e} \Rightarrow \mathsf{aborts}\ \kappa}$$

Figure 4.7: Expression sequence $\vec{e}$ evaluates left to right to outcome o under environment $\rho : \Gamma$

$\rho, e \Rightarrow o$ (continued)

eval-binop-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e \odot e' \Rightarrow \text{aborts } \kappa}$$

eval-and-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e \texttt{ and } e' \Rightarrow \text{aborts } \kappa}$$

eval-or-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e \texttt{ or } e' \Rightarrow \text{aborts } \kappa}$$

eval-cond-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e_1 \texttt{ if } e \texttt{ else } e_2 \Rightarrow \text{aborts } \kappa}$$

eval-attr-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e.x \Rightarrow \text{aborts } \kappa}$$

eval-subscript-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e[e'] \Rightarrow \text{aborts } \kappa}$$

eval-call-fail
$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, e(e_1, \ldots, e_n) \Rightarrow \text{aborts } \kappa}$$

eval-call-fail-args
$$\frac{\rho, e \Rightarrow \text{val } v \qquad \rho, \vec{e} \Rightarrow \text{aborts } \kappa}{\rho, e(e_1, \ldots, e_n) \Rightarrow \text{aborts } \kappa}$$

eval-list-fail
$$\frac{\rho, e_1 \cdot \ldots \cdot e_n \Rightarrow \text{aborts } \kappa}{\rho, [e_1, \ldots, e_n] \Rightarrow \text{aborts } \kappa}$$

eval-tuple-fail
$$\frac{\rho, e_1 \cdot \ldots \cdot e_n \Rightarrow \text{aborts } \kappa}{\rho, (e_1, \ldots, e_n) \Rightarrow \text{aborts } \kappa}$$

eval-dict-fail
$$\frac{\rho, e_1 \cdot e_1' \cdot \ldots \cdot e_n \cdot e_n' \Rightarrow \text{aborts } \kappa}{\rho, \{e_1 : e_1', \ldots, e_n : e_n'\} \Rightarrow \text{aborts } \kappa}$$

eval-list-comp-quals-fail
$$\frac{\rho, \vec{g} \Rightarrow \text{aborts } \kappa}{\rho, [\, e \;\; g_1 \;\; \ldots \;\; g_n] \Rightarrow \text{aborts } \kappa}$$

eval-list-comp-fail
$$\frac{\rho, \vec{g} \Rightarrow \text{val } (\rho_1 \cdot \ldots \cdot \rho_m) \qquad \rho_i, e \Rightarrow \text{val } v_i \quad (\forall i < j) \qquad \rho_j, e \Rightarrow \text{aborts } \kappa}{\rho, [\, e \;\; g_1 \;\; \ldots \;\; g_n] \Rightarrow \text{aborts } \kappa}$$

eval-dict-comp-fail-quals
$$\frac{\rho, \vec{g} \Rightarrow \text{aborts } \kappa}{\rho, \{e : e' \;\; g_1 \;\; \ldots \;\; g_n\} \Rightarrow \text{aborts } \kappa}$$

eval-dict-comp-fail-key
$$\frac{\rho, \vec{g} \Rightarrow \text{val } (\rho_1 \cdot \ldots \cdot \rho_m) \qquad \rho_i, e \Rightarrow \text{val } w_i \quad \rho_i, e' \Rightarrow \text{val } v_i \quad (\forall i < j) \qquad \rho_j, e \Rightarrow \text{aborts } \kappa}{\rho, \{e : e' \;\; g_1 \;\; \ldots \;\; g_n\} \Rightarrow \text{aborts } \kappa}$$

eval-dict-comp-fail-value
$$\frac{\rho, \vec{g} \Rightarrow \text{val } (\rho_1 \cdot \ldots \cdot \rho_m) \qquad \rho_i, e \Rightarrow \text{val } w_i \quad \rho_i, e' \Rightarrow \text{val } v_i \quad (\forall i < j) \qquad \rho_j, e \Rightarrow \text{val } w_j \qquad \rho_j, e' \Rightarrow \text{aborts } \kappa}{\rho, \{e : e' \;\; g_1 \;\; \ldots \;\; g_n\} \Rightarrow \text{aborts } \kappa}$$

eval-constr-fail
$$\frac{\rho, e_1 \cdot \ldots \cdot e_{n+m} \Rightarrow \text{aborts } \kappa}{\rho, q(e_1, \ldots, e_n, y_1{=}e_{n+1}, \ldots, y_m{=}e_{n+m}) \Rightarrow \text{aborts } \kappa}$$

Figure 4.8: Expression evaluation, continued (propagation from subexpressions)

$\rho, e \Rightarrow o$ (continued)

eval-attr-missing

$$\frac{\rho, e \Rightarrow \text{val } \mathsf{Obj}_\Delta(C, \rho') \qquad x \notin \mathrm{dom}(\rho')}{\rho, e.x \Rightarrow \text{aborts AttributeError}}$$

eval-call-lambda-arity

$$\frac{\rho, e \Rightarrow \text{val } \mathsf{Lam}_\Delta(\rho', \vec{x}, e') \qquad \rho, \vec{e} \Rightarrow \text{val } \vec{u} \qquad |\vec{x}| \neq |\vec{u}|}{\rho, e(e_1, \ldots, e_n) \Rightarrow \text{aborts TypeError}}$$

eval-call-def-arity

$$\frac{\rho, e \Rightarrow \text{val } \mathsf{Def}_\Delta(\rho', \vec{d}, j) \qquad d_j = \texttt{def } f_j(\vec{x}_j\, \vec{\pi}_j)\, \omega_j \colon s_j \qquad \rho, \vec{e} \Rightarrow \text{val } \vec{u} \qquad |\vec{x}_j| \neq |\vec{u}|}{\rho, e(e_1, \ldots, e_n) \Rightarrow \text{aborts TypeError}}$$

eval-call-nonfun

$$\frac{\rho, e \Rightarrow \text{val } v \qquad \rho, \vec{e} \Rightarrow \text{val } \vec{u} \qquad v \text{ neither closure nor primitive}}{\rho, e(e_1, \ldots, e_n) \Rightarrow \text{aborts TypeError}}$$

Figure 4.9: Expression evaluation, continued (aborting cases)

$\rho, \vec{g} \Rightarrow o$

eval-quals-nil

$$\frac{}{\rho, \varepsilon \Rightarrow \text{val } \rho}$$

eval-qual-guard-true

$$\frac{\rho, e \Rightarrow \text{val } \texttt{True} \qquad \rho, \vec{g} \Rightarrow o}{\rho, \texttt{if } e \cdot \vec{g} \Rightarrow o}$$

eval-qual-guard-false

$$\frac{\rho, e \Rightarrow \text{val } \texttt{False}}{\rho, \texttt{if } e \cdot \vec{g} \Rightarrow \text{val } \varepsilon}$$

eval-qual-guard-fail

$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, \texttt{if } e \cdot \vec{g} \Rightarrow \text{aborts } \kappa}$$

eval-qual-generator

$$\frac{\rho, e \Rightarrow \text{val } v \qquad \mathrm{iter}(v) = u_1 \cdot \ldots \cdot u_n \qquad \rho \otimes \{x \colon u_i\}, \vec{g} \Rightarrow \text{val } \vec{\rho}_i \quad (\forall i)}{\rho, \texttt{for } x \texttt{ in } e \cdot \vec{g} \Rightarrow \text{val } (\vec{\rho}_1 +\!\!+ \ldots +\!\!+ \vec{\rho}_n)}$$

eval-qual-generator-fail

$$\frac{\rho, e \Rightarrow \text{aborts } \kappa}{\rho, \texttt{for } x \texttt{ in } e \cdot \vec{g} \Rightarrow \text{aborts } \kappa}$$

eval-qual-generator-nonseq

$$\frac{\rho, e \Rightarrow \text{val } v \qquad \mathrm{iter}(v) \text{ undefined}}{\rho, \texttt{for } x \texttt{ in } e \cdot \vec{g} \Rightarrow \text{aborts TypeError}}$$

eval-qual-generator-fail-body

$$\frac{\rho, e \Rightarrow \text{val } v \qquad \mathrm{iter}(v) = u_1 \cdot \ldots \cdot u_n \qquad \rho \otimes \{x \colon u_i\}, \vec{g} \Rightarrow \text{val } \vec{\rho}_i \quad (\forall i < j) \qquad \rho \otimes \{x \colon u_j\}, \vec{g} \Rightarrow \text{aborts } \kappa}{\rho, \texttt{for } x \texttt{ in } e \cdot \vec{g} \Rightarrow \text{aborts } \kappa}$$

Figure 4.10: Comprehension qualifiers $\vec{g}$ evaluate to outcome o under environment $\rho : \Gamma$

ρ, x imports o

eval-imp-member

$$\frac{x \colon v \in \rho \qquad v \neq \mathsf{Mod}(q') \quad (\forall q')}{\rho, x \text{ imports val } v}$$

eval-imp-submodule

$$\frac{x \colon \mathsf{Mod}(q') \in \rho \qquad q' \Rightarrow_{\mathsf{load}} \text{val } \rho'}{\rho, x \text{ imports val } \mathsf{Mod}(q', \rho')}$$

eval-imp-submodule-fail

$$\frac{x \colon \mathsf{Mod}(q') \in \rho \qquad q' \Rightarrow_{\mathsf{load}} \text{aborts } \kappa}{\rho, x \text{ imports aborts } \kappa}$$

Figure 4.11: Name x imported from module environment ρ has outcome o

$\vec{\imath} \Rightarrow o$

eval-imports-nil

$$\frac{}{\varepsilon \Rightarrow \mathsf{val}\ \emptyset}$$

eval-import

$$\frac{q' = x \cdot \vec{x} \qquad q' \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho \qquad q', \mathsf{Mod}(q', \rho)\ \mathsf{loads\text{-}as}\ v \qquad \vec{\imath} \Rightarrow o}{\texttt{import}\ q' \cdot \vec{\imath} \Rightarrow \mathsf{let}\ \rho' \leftarrow o\ \mathsf{in}\ \mathsf{val}\ (\{x : v\} \lhd \rho')}$$

eval-import-fail

$$\frac{q' = x \cdot \vec{x} \qquad q' \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}{\texttt{import}\ q' \cdot \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}$$

eval-from-import

$$\frac{q' \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho \qquad q^\dagger \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho_{q^\dagger} \quad (\forall q^\dagger \prec q',\ q^\dagger \not\preceq q) \qquad \rho, x_i\ \mathsf{imports\ val}\ v_i \quad (\forall i) \qquad \vec{\imath} \Rightarrow o}{\texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} \Rightarrow \mathsf{let}\ \rho^\dagger \leftarrow o\ \mathsf{in}\ \mathsf{val}\ (\{x_1 : v_1, \ldots, x_{n+1} : v_{n+1}\} \lhd \rho^\dagger)}$$

eval-from-import-fail

$$\frac{q' \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}{\texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}$$

eval-from-import-fail-ancestor

$$\frac{q' \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho \qquad q^\dagger \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa \quad (\text{some } q^\dagger \prec q',\ q^\dagger \not\preceq q)}{\texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}$$

eval-from-import-fail-member

$$\frac{\begin{array}{l} q' \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho \\ q^\dagger \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho_{q^\dagger} \quad (\forall q^\dagger \prec q',\ q^\dagger \not\preceq q) \qquad \rho, x_i\ \mathsf{imports\ val}\ v_i \quad (\forall i < j) \qquad \rho, x_j\ \mathsf{imports\ aborts}\ \kappa \end{array}}{\texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}$$

(a) Import prefix $\vec{\imath}$ of module q evaluates to outcome o

$q, v\ \mathsf{loads\text{-}as}\ v'$

eval-loads-as-simple

$$\frac{}{x, v\ \mathsf{loads\text{-}as}\ v}$$

eval-loads-as-qualified

$$\frac{q \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho \qquad q, \mathsf{Mod}(q, \rho \lhd \{x : v\})\ \mathsf{loads\text{-}as}\ v'}{q.x, v\ \mathsf{loads\text{-}as}\ v'}$$

(b) Reference v for module q loads as the reference v' for the root of q, loading the ancestors of q

Figure 4.12: Import evaluation

$q \Rightarrow_{\mathsf{load}} o$

eval-predefined
$$\frac{\mathcal{P}(q) = (\rho, \Gamma)}{q \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho}$$

eval-module
$$\frac{\mathcal{M}(q) = \vec{\imath}\ t \qquad \vec{\imath} \Rightarrow \mathsf{val}\ \rho \qquad \mathcal{P}(\texttt{builtins}) = (\rho', \Gamma') \qquad \rho' \otimes \rho, t \Rightarrow \mathsf{assigns}\ \rho^\dagger}{q \Rightarrow_{\mathsf{load}} \mathsf{val}\ (\mathrm{submods}(q) \otimes \rho^\dagger)}$$

eval-module-fail
$$\frac{\mathcal{M}(q) = \vec{\imath}\ t \qquad \vec{\imath} \Rightarrow \mathsf{aborts}\ \kappa}{q \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}$$

eval-module-fail-body
$$\frac{\mathcal{M}(q) = \vec{\imath}\ t \qquad \vec{\imath} \Rightarrow \mathsf{val}\ \rho \qquad \mathcal{P}(\texttt{builtins}) = (\rho', \Gamma') \qquad \rho' \otimes \rho, t \Rightarrow \mathsf{aborts}\ \kappa}{q \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}$$

$\mathcal{M} \Rightarrow \kappa$

eval-program
$$\frac{\texttt{__main__} \Rightarrow_{\mathsf{load}} \mathsf{val}\ \rho}{\mathcal{M} \Rightarrow \mathsf{SystemExit}(0)}$$

eval-program-abort
$$\frac{\texttt{__main__} \Rightarrow_{\mathsf{load}} \mathsf{aborts}\ \kappa}{\mathcal{M} \Rightarrow \kappa}$$

Figure 4.13: Module and program evaluation

Annex A

Auxiliary functions (normative)

This annex is normative. It defines the auxiliary functions used by the well-formedness rules (Chapter 3) and the operational semantics (Chapter 4).

A.1 Syntactic analyses

The syntactic analyses introduced in §3.1 are defined by their principal cases; the remaining cases are stated with each.

assigns(t) — *set of variables*

The variables assigned anywhere in t, not descending into nested functions, which have their own scope. It is empty on the statements not shown and is the union over the branches of a conditional, and on a module body $\vec{\imath}\ t$ acts on t.

$$\begin{aligned}
\text{assigns}(x\pi = e) &= \{x\} \\
\text{assigns}(t\ t') &= \text{assigns}(t) \cup \text{assigns}(t') \\
\text{assigns}(\vec{d}) &= \{\vec{f}\} \quad \text{where } d_i = \texttt{def}\ f_i(\vec{x}_i\ \vec{\pi}_i)\ \omega_i\texttt{:}\ s_i \\
\text{assigns}(\texttt{match}\ e\texttt{:}\ \texttt{case}\ p_1\texttt{:}\ b_1\ \ldots\ \texttt{case}\ p_{n+1}\texttt{:}\ b_{n+1}) &= \textstyle\bigcup_i \left(\text{binds}(p_i) \cup \text{assigns}(b_i)\right) \\
\text{assigns}(\texttt{@dataclass class}\ c\ \texttt{:}\ \ \phi) &= \{c\} \\
\text{assigns}(\texttt{@dataclass class}\ c(c')\ \texttt{:}\ \ \phi) &= \{c\}
\end{aligned}$$

binds(p) — *set of variables*

The variables introduced by pattern p, or by the generators of qualifier sequence $\vec{g}$. On the cases not shown it recurses structurally, taking the union over the immediate subpatterns.

$$\begin{aligned}
\text{binds}(x) &= \{x\} \\
\text{binds}(p\ \texttt{as}\ x) &= \text{binds}(p) \cup \{x\} \\
\text{binds}(\varepsilon) &= \emptyset \\
\text{binds}(\texttt{if}\ e \cdot \vec{g}) &= \text{binds}(\vec{g}) \\
\text{binds}(\texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g}) &= \{x\} \cup \text{binds}(\vec{g})
\end{aligned}$$

captures(t) — *set of variables*

The variables of the enclosing scope captured by lambdas and nested function definitions within t, or by lambdas within the qualifier sequence $\vec{g}$ of a comprehension. On the cases not shown it takes

the union of captures_e over the subexpressions of t and of captures over its sub-statements, and on a module body acts as assigns does.

$$\begin{aligned}
\text{captures}(t\ t') &= \text{captures}(t) \cup \text{captures}(t') \\
\text{captures}(\vec{d}) &= \textstyle\bigcup_i \left(\text{fv}(b_i) \setminus (\{\vec{x}_i\} \cup \text{assigns}(b_i))\right) \setminus \{\vec{f}\} \\
&\quad \text{where } d_i = \texttt{def}\ f_i(\vec{x}_i\ \vec{\pi}_i)\ \omega_i\texttt{:}\ s_i \\
\text{captures}(\texttt{match}\ e\texttt{:}\ \texttt{case}\ p_1\texttt{:}\ b_1\ \ldots\ \texttt{case}\ p_{n+1}\texttt{:}\ b_{n+1}) &= \text{captures}_\text{e}(e) \cup \textstyle\bigcup_i \left(\text{captures}(b_i) \setminus \text{binds}(p_i)\right) \\
\text{captures}(\varepsilon) &= \emptyset \\
\text{captures}(\texttt{if}\ e \cdot \vec{g}) &= \text{captures}_\text{e}(e) \cup \text{captures}(\vec{g}) \\
\text{captures}(\texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g}) &= \text{captures}_\text{e}(e) \cup (\text{captures}(\vec{g}) \setminus \{x\})
\end{aligned}$$

$\text{captures}_\text{e}(e)$	*set of variables*

The variables of the enclosing scope captured by lambdas within expression e; the generator variables of a comprehension are local to it. The subscript distinguishes this from $\text{captures}(t)$, since an expression statement is both an expression and a statement. On the cases not shown it recurses structurally, taking the union over the immediate subexpressions.

$$\begin{aligned}
\text{captures}_\text{e}(\texttt{lambda}\ \vec{x}\texttt{:}\ e) &= \text{fv}_\text{e}(e) \setminus \{\vec{x}\} \\
\text{captures}_\text{e}([\ e\ \vec{g}\ ]) &= \text{captures}(\vec{g}) \cup (\text{captures}_\text{e}(e) \setminus \text{binds}(\vec{g})) \\
\text{captures}_\text{e}(\{e\texttt{:}\ e'\ \vec{g}\}) &= \text{captures}(\vec{g}) \cup ((\text{captures}_\text{e}(e) \cup \text{captures}_\text{e}(e')) \setminus \text{binds}(\vec{g}))
\end{aligned}$$

$\text{fv}(s)$	*set of variables*

The free variables of statement s, defined in the usual way.

$\text{fv}_\text{e}(e)$	*set of variables*

The free variables of expression e, defined in the usual way.

A.2 Classes and modules

The functions on module names, field lists and class entries introduced in §2.3, and the resolution of qualified names in an environment.

$\text{submods}(q)$	*context*

The context that binds each submodule of q to a stub: an entry $x : \mathsf{Mod}(q.x)$ for each module $q.x$ of the program or of the predefined modules.

$$\text{submods}(q) = \{x : \mathsf{Mod}(q.x) \mid q.x \in \text{dom}(\mathcal{M}) \cup \text{dom}(\mathcal{P})\}$$

own-fields(ϕ) *sequence of variables*

The field names a field list declares, in order.

$$\text{own-fields}(x_1 : \tau_1 \ \ldots \ x_{n+1} : \tau_{n+1}) = x_1, \ldots, x_{n+1}$$
$$\text{own-fields}(\texttt{pass}) = \varepsilon$$

fields(C) *sequence of variables*

The fields of class C: the fields of its base class, if any, followed by the fields C declares.

$$\text{fields}(\mathsf{Cls}_\Gamma(q.c, \vec{x}, \bot)) = \vec{x}$$
$$\text{fields}(\mathsf{Cls}_\Gamma(q.c, \vec{x}, c')) = \text{fields}(\Gamma(c')) +\!\!+ \vec{x}$$

ancestors(C) *set of class entries*

Class C together with all its base classes.

$$\text{ancestors}(\mathsf{Cls}_\Gamma(q.c, \vec{x}, \bot)) = \{\mathsf{Cls}_\Gamma(q.c, \vec{x}, \bot)\}$$
$$\text{ancestors}(\mathsf{Cls}_\Gamma(q.c, \vec{x}, c')) = \{\mathsf{Cls}_\Gamma(q.c, \vec{x}, c')\} \cup \text{ancestors}(\Gamma(c'))$$

field-map($C, \vec{u}$) *map from variables*

The argument supplied for each field of C: positional arguments fill the fields in order and keyword arguments match by name. It is undefined unless the arguments cover the fields exactly, so constructor patterns and expressions check and evaluate field by field without regard to how a constructor splits its arguments.

$$\text{field-map}(C, (u_1, \ldots, u_n, y_1{=}u_{n+1}, \ldots, y_m{=}u_{n+m})) = \{x_1 : u_1, \ldots, x_n : u_n, y_1 : u_{n+1}, \ldots, y_m : u_{n+m}\}$$
$$\text{if fields}(C) = x_1 \cdot \ldots \cdot x_n \cdot \vec{y}\,' \text{ and } \{\vec{y}\} = \{\vec{y}\,'\}$$

env(ρ, q) *environment*

The environment of module q as loaded in ρ, reached by following the module references along q.

$$\text{env}(\rho, x) = \rho' \qquad \text{where } \rho(x) = \mathsf{Mod}(q', \rho')$$
$$\text{env}(\rho, q.x) = \rho' \qquad \text{where } \text{env}(\rho, q)(x) = \mathsf{Mod}(q', \rho')$$

resolve-class(ρ, q) *class entry*

The class entry that qualified name q denotes in ρ. A simple class name is looked up in ρ directly; a qualified name is looked up in the environment of its module.

$$\text{resolve-class}(\rho, c) = \rho(c)$$
$$\text{resolve-class}(\rho, q.c) = \text{env}(\rho, q)(c)$$

name(q) *string*

The string literal naming module q, so that name(`__main__`) = `"__main__"`.

A.3 Values

The functions on values used by the evaluation rules. The comparisons `==`, `!=`, `in` and `not in` are given in terms of structural equality eq and membership contains. Where defined, the operators agree with Python, yielding aborts κ where Python raises an exception. Where undefined, evaluation of the expression has no derivation, and an implementation may abort or produce Python's result (Chapter 1). Because equality is partial, the order in which elements are compared is observable: equality and membership short-circuit at the first element that decides the result, so an undefined comparison later in a sequence does not make the whole comparison undefined.

$\hat{\odot}(v, v')$ *outcome*

The meaning of a strict binary operator.

$\odot$	$\hat{\odot}(v, v')$
`==`	val `False` if both operands are `nan`, and val eq(v, v') otherwise
`!=`	val `True` if both operands are `nan` or eq$(v, v') =$ `False`, and val `False` if eq$(v, v') =$ `True`
`in`	val contains(v', v)
`not in`	val `True` if contains$(v', v) =$ `False`, and val `False` if contains$(v', v) =$ `True`
`<` `<=` `>` `>=`	[**TODO:** Ordering, as in Python]
`+` `-` `*` `/` `//` `%` `**`	[**TODO:** Arithmetic, as in Python. Float precision is left to the implementation. aborts ZeroDivisionError where the divisor is zero, and aborts TypeError where an operand is not a number]

$\hat{\ominus}(v)$ *outcome*

The meaning of a unary operator.

$\ominus$	$\hat{\ominus}(v)$
`not`	val `True` if $v =$ `False`, and val `False` if $v =$ `True`
`+`	val v if v is a number, and aborts TypeError otherwise
`-`	val $-v$ if v is a number, and aborts TypeError otherwise

eq(v, v') *Boolean*

Structural equality: literals by value, lists and tuples by their elements, dictionaries with the same keys by their values, and objects of the same class by their field values, each through eq-elems. `None` compares with any value; otherwise equality is undefined between values of different kinds. Dictionaries are compared in the entry order of the left operand.

$$
\begin{aligned}
\text{eq}(\ell, \ell) &= \texttt{True} && \text{if } \ell \text{ is not } \texttt{nan} \\
\text{eq}(n, n') &= \texttt{False} && \text{if } n \neq n' \\
\text{eq}(w, w') &= \texttt{False} && \text{if } w \neq w' \\
\text{eq}(\texttt{None}, v) &= \texttt{False} && \text{if } v \neq \texttt{None} \\
\text{eq}(v, \texttt{None}) &= \texttt{False} && \text{if } v \neq \texttt{None} \\
\text{eq}(\texttt{True}, \texttt{False}) &= \texttt{False} \\
\text{eq}(\texttt{False}, \texttt{True}) &= \texttt{False}
\end{aligned}
$$

$$\text{eq}([v_1, \ldots, v_n], [v'_1, \ldots, v'_m]) = \text{eq-elems}(v_1 \cdot \ldots \cdot v_n, v'_1 \cdot \ldots \cdot v'_m)$$
$$\text{eq}((v_1, \ldots, v_n), (v'_1, \ldots, v'_m)) = \text{eq-elems}(v_1 \cdot \ldots \cdot v_n, v'_1 \cdot \ldots \cdot v'_m)$$

$$\begin{aligned} \text{eq}(\delta, \delta') &= \text{eq-elems}(v_1 \cdot \ldots \cdot v_n, \delta'(w_1) \cdot \ldots \cdot \delta'(w_n)) && \text{if } \text{dom}(\delta) = \text{dom}(\delta') \\ &\quad \text{where } \delta = w_1 : v_1 \cdot \ldots \cdot w_n : v_n \\ \text{eq}(\delta, \delta') &= \texttt{False} && \text{if } \text{dom}(\delta) \neq \text{dom}(\delta') \end{aligned}$$

$$\begin{aligned} \text{eq}(\mathsf{Obj}_\Gamma(C, \rho), \mathsf{Obj}_{\Gamma'}(C, \rho')) &= \text{eq-elems}(\rho(x_1) \cdot \ldots \cdot \rho(x_n), \rho'(x_1) \cdot \ldots \cdot \rho'(x_n)) \\ &\quad \text{if } \text{fields}(C) = x_1, \ldots, x_n \\ \text{eq}(\mathsf{Obj}_\Gamma(C, \rho), \mathsf{Obj}_{\Gamma'}(C', \rho')) &= \texttt{False} && \text{if } C \neq C' \end{aligned}$$

eq-elems($\vec{v}, \vec{v}'$) — *Boolean*

Equality of two sequences of values, compared from the first element: `False` at the first unequal pair or if the lengths differ, and `True` when every pair is equal.

$$\begin{aligned} \text{eq-elems}(\varepsilon, \varepsilon) &= \texttt{True} \\ \text{eq-elems}(v \cdot \vec{v}, v' \cdot \vec{v}') &= \texttt{False} && \text{if } \text{eq}(v, v') = \texttt{False} \\ \text{eq-elems}(v \cdot \vec{v}, v' \cdot \vec{v}') &= \text{eq-elems}(\vec{v}, \vec{v}') && \text{if } \text{eq}(v, v') = \texttt{True} \\ \text{eq-elems}(\vec{v}, \vec{v}') &= \texttt{False} && \text{if } |\vec{v}| \neq |\vec{v}'| \end{aligned}$$

contains(v, v') — *Boolean*

Whether v' is an element of v: of a list or tuple by structural equality, of a dictionary as a key, and of a string as a substring.

$$\text{contains}([v_1, \ldots, v_n], v) = \text{contains-elems}(v_1 \cdot \ldots \cdot v_n, v)$$
$$\text{contains}((v_1, \ldots, v_n), v) = \text{contains-elems}(v_1 \cdot \ldots \cdot v_n, v)$$

$$\begin{aligned} \text{contains}(\delta, w) &= \texttt{True} && \text{if } w \in \text{dom}(\delta) \\ \text{contains}(\delta, w) &= \texttt{False} && \text{if } w \notin \text{dom}(\delta) \end{aligned}$$

$$\begin{aligned} \text{contains}(w', w) &= \texttt{True} && \text{if } w' = w_1 \mathbin{+\!\!+} w \mathbin{+\!\!+} w_2 \text{ for some } w_1, w_2 \\ \text{contains}(w', w) &= \texttt{False} && \text{(otherwise)} \end{aligned}$$

contains-elems($\vec{v}, v$) — *Boolean*

Whether a sequence of values has an element structurally equal to v, taking the elements in order and stopping at the first equal one.

$$\begin{aligned} \text{contains-elems}(\varepsilon, v) &= \texttt{False} \\ \text{contains-elems}(v' \cdot \vec{v}, v) &= \texttt{True} && \text{if } \text{eq}(v, v') = \texttt{True} \\ \text{contains-elems}(v' \cdot \vec{v}, v) &= \text{contains-elems}(\vec{v}, v) && \text{if } \text{eq}(v, v') = \texttt{False} \end{aligned}$$

elems(v) *sequence of values*

The elements of a list or tuple, or the characters of a string, in order.

$$\begin{aligned}
\text{elems}([v_1, \ldots, v_n]) &= v_1 \cdot \ldots \cdot v_n \\
\text{elems}((v_1, \ldots, v_n)) &= v_1 \cdot \ldots \cdot v_n \\
\text{elems}(w) &= w_1 \cdot \ldots \cdot w_m \qquad \text{where } w = w_1 \mathbin{+\!\!+} \ldots \mathbin{+\!\!+} w_m, \text{ each } w_i \text{ one character}
\end{aligned}$$

iter(v) *sequence of values*

The sequence a generator draws from v: the keys of a dictionary, and otherwise its elements.

$$\begin{aligned}
\text{iter}(\delta) &= w_1 \cdot \ldots \cdot w_n \qquad \text{where } \delta = w_1 : v_1 \cdot \ldots \cdot w_n : v_n \\
\text{iter}(v) &= \text{elems}(v) \qquad \text{(otherwise)}
\end{aligned}$$

getitem(v, v') *outcome*

Subscripting: the value under key v' of a dictionary, or the element at index v' of a sequence, counting from the end when the index is negative. It aborts with KeyError, IndexError or TypeError where Python raises the exception of that name.

$$\begin{aligned}
\text{getitem}(\delta, w) &= \mathsf{val}\ \delta(w) && \text{if } w \in \text{dom}(\delta) \\
\text{getitem}(\delta, w) &= \mathsf{aborts\ KeyError} && \text{if } w \notin \text{dom}(\delta) \\
\text{getitem}(v, n) &= \mathsf{val}\ v_{n+1} && \text{if elems}(v) = v_1 \cdot \ldots \cdot v_m \text{ and } 0 \le n < m \\
\text{getitem}(v, n) &= \mathsf{val}\ v_{m+n+1} && \text{if elems}(v) = v_1 \cdot \ldots \cdot v_m \text{ and } -m \le n < 0 \\
\text{getitem}(v, n) &= \mathsf{aborts\ IndexError} && \text{if elems}(v) = v_1 \cdot \ldots \cdot v_m \text{ and } (n < -m \text{ or } n \ge m) \\
\text{getitem}(v, v') &= \mathsf{aborts\ TypeError} && \text{(otherwise; } v \text{ not a dictionary)}
\end{aligned}$$

outcome(r) *outcome*

The result of a statement read as the outcome of calling it: a returned value, `None` for a statement that completes without returning, and an abort unchanged.

$$\begin{aligned}
\text{outcome}(\mathsf{returns}\ v) &= \mathsf{val}\ v \\
\text{outcome}(\mathsf{assigns}\ \rho) &= \mathsf{val}\ \texttt{None} \\
\text{outcome}(\mathsf{aborts}\ \kappa) &= \mathsf{aborts}\ \kappa
\end{aligned}$$

dispatch($\rho, v, \vec{p}, \vec{s}$) *bindings and statement*

The case a `match` statement takes for value v: the bindings and statement of the first pattern that matches, or no bindings and `pass` if none does.

$$\begin{aligned}
\text{dispatch}(\rho, v, \varepsilon, \varepsilon) &= (\varnothing, \texttt{pass}) \\
\text{dispatch}(\rho, v, p \cdot \vec{p}, s \cdot \vec{s}) &= (\rho', s) && \text{if } \rho, p, v \rightsquigarrow \mathsf{match}\ \rho' \\
\text{dispatch}(\rho, v, p \cdot \vec{p}, s \cdot \vec{s}) &= \text{dispatch}(\rho, v, \vec{p}, \vec{s}) && \text{if } \rho, p, v \rightsquigarrow \mathsf{no\text{-}match}
\end{aligned}$$

$\mathrm{entries}(\delta, \delta')$	*dictionary entries*

The entries δ extended by those of δ' in order, each replacing any earlier entry under the same key.

$$\begin{aligned} \mathrm{entries}(\delta, \varepsilon) &= \delta \\ \mathrm{entries}(\delta, w : v \cdot \delta') &= \mathrm{entries}(\mathrm{update}(\delta, w, v), \delta') \end{aligned}$$

$\mathrm{update}(\delta, w, v)$	*dictionary entries*

The entries δ with w bound to v, replacing any earlier binding of w.

$$\begin{aligned} \mathrm{update}(\varepsilon, w, v) &= w : v \\ \mathrm{update}(w : u \cdot \delta, w, v) &= w : v \cdot \delta \\ \mathrm{update}(w' : u \cdot \delta, w, v) &= w' : u \cdot \mathrm{update}(\delta, w, v) && \text{if } w \neq w' \end{aligned}$$

Annex B

Excluded Python behaviours (informative)

This annex is informative and imposes no requirements of its own. It collects the Python behaviours PurePy excludes and the stage at which each exclusion takes effect; the normative basis is the well-formedness rules (Chapter 3) and the operational semantics (Chapter 4).

B.1 Mutable state

Figure B.1 lists the Python constructs associated with mutation and shared state, and says how and when PurePy excludes each.

Construct	Python	Exclusion	Mechanism
`global` and `nonlocal`	Direct assignments to variable of enclosing scope	Syntactic	Not in grammar
`del`	Removes binding	Syntactic	Not in grammar
Augmented assignment `+=`, etc.	Updates in place where type supports it	Syntactic	Not in grammar
Walrus operator `:=`	Assigns within expression	Syntactic	Not in grammar
Item assignment `xs[0] = v`	Updates container in place	Syntactic	Not in grammar; assignment targets are variables
Attribute assignment `p.x = v`	Updates object in place	Syntactic	Not in grammar; assignment targets are variables
Mutating methods, such as `list.append`	Update value in place	Dynamic	Attribute reference undefined
Reassignment of captured variable	Closure sees new value	Static	Variable captured by statement may not be assigned later in sequence, nor by statement itself
Lambdas capturing comprehension variable	Variable mutated on each iteration; lambda sees final value	Static	Lambda may not capture generator variable

Figure B.1: Python mutation of state excluded by PurePy

B.2 Implicit coercions

Python accepts several implicit coercions and lax matches. Figure B.2 lists those PurePy excludes, and records at which stage each exclusion takes effect. A dynamically excluded operation leaves the run without a result, so the program runs under Python but not under PurePy.

Construct	Python	Exclusion	Mechanism
Tests of `if` and `assert`	Coerced to `bool`	Dynamic	Must already be Boolean
Sequence patterns	List patterns match tuples, and tuple patterns lists	Dynamic	List patterns match only lists, tuple patterns only tuples
Constructor patterns	Positional class patterns may supply fewer sub-patterns than class has fields	Static	Require exact arity
Dictionary keys	Any hashable value, with `1`, `1.0` and `True` equivalent as keys	Static in patterns, dynamic in expressions	Strings only
`assert` message	Any object, shown in traceback	Dynamic	Strings only
`==` and `!=`	Compare any two operands, giving `False` when unrelated	Dynamic	Undefined between, say, number and string, or list and tuple; only `None` compares with anything
Functions under `==`	Compared by identity, so two textually identical lambdas are unequal	Dynamic	Equality undefined
`nan` comparison via container equality	Elements compared by identity before equality, so identical NaNs match but distinct ones do not	Dynamic	Undefined when both elements are `nan`

Figure B.2: Python coercions and lax matches excluded by PurePy

B.3 Runtime errors

A well-formed PurePy program avoids the Python runtime errors listed in Figure B.3, under the conditions noted there. Well-formedness does not prevent errors from ill-typed operations, such as wrong operand types, calling a non-function, calling a function with the wrong number of arguments, or attribute access on an object lacking the attribute. Each of these has a termination kind and a rule in the semantics (Figure 4.9).

Python error	Ruled out when	See
`NameError`, `UnboundLocalError`	Always: a variable may be referenced only where it is definitely assigned.	Figure 3.1, Figure 3.2
`ModuleNotFoundError`	Always: an import requires its target to be a module of the program.	Figure 3.6
`ImportError`	Always: a from-import requires each imported name to be a member of the source module, and import cycles are rejected. (A Python cycle may run or fail, depending on where the cycle is entered.)	Figure 3.5, Figure 3.7
`AttributeError`	The reference is a module attribute, which must resolve to a definitely assigned member.	Figure 3.2
`TypeError`	The operation is a constructor call or class pattern, which must match the class's arity exactly. (Only excess sub-patterns are a Python error; a pattern with fewer is excluded even though Python permits it.)	Figure 3.2, Figure 3.4

Figure B.3: Python errors prevented by well-formedness

Bibliography

[1] James Gosling, Bill Joy, Guy L. Steele Jr., Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman. *The Java Language Specification, Java SE 24 Edition*. Oracle, 2025. URL https://docs.oracle.com/javase/specs/jls/se24/html/index.html.

[2] W. G. van Hoorn and B. van Rootselaar. Fundamental notions in the theory of seminearrings. *Compositio Mathematica*, 18(1-2):65–78, 1967. URL https://www.numdam.org/item/CM_1967__18_1-2_65_0/.